OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date:

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level) **Mandatory Execution and GitHub Submission**

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator). • Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted. • Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/rakshanasudhakar1905/CSA0409-Operating-Systems/tree/main/ASSIGNMENT

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Processes and Process States 6 Marks Bloom's Level: K5 – Evaluate

Problem Context – Online Examination Server

During an online examination, the server runs a separate process for every student's session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the server becomes slow and some sessions appear to "hang".

Your Challenge

Draw the process-state diagram (new, ready, running, waiting, terminated) and the process control block (PCB). Write a program that creates child processes with fork() and observe them with ps and /proc.

Evaluation

Evaluate where the time goes when 2,000 processes are active: context-switch cost, memory per process, and time spent waiting for I/O. Judge whether a separate process per student is a sound design.

Design Decision

Decide between one process per student, one thread per student and a fixed pool of worker processes, and defend your decision.

Alternative Concept and Complexity Analysis

If each student session were a thread inside one server process, instead of a separate process, what would the solution be? Evaluate the context-switch time and the memory per session (a few KB of stack versus a full address space), and the effect of one session crashing.

Evidence Expected

A state diagram, a PCB description, a working fork() program with ps evidence and a defended design decision.

Q2 – CPU Scheduling 8 Marks Bloom's Level: K5 – Evaluate

Problem Context – Cloud Video-Encoding Service

A video service receives short jobs (thumbnails, 2 seconds) and long jobs (full films, 20 minutes) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used.

Your Challenge

Compute the Gantt chart, waiting time, turnaround time and response time for the same set of jobs under FCFS, SJF, SRTF, priority and Round Robin scheduling. Implement Round Robin.

Evaluation

Evaluate how the time quantum affects Round Robin (try 1, 3, 5 and 20), the risk of starvation in SJF and priority scheduling, and how ageing removes it. Explain how a multilevel feedback queue treats short and long jobs.

Design Decision

Recommend the scheduling policy for the video service and justify it with your calculated averages.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define N 4

int main(void)
{const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int quantum = 3;

    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];
    int time = 0, done = 0;
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while (done < N) {
```

```
        int p = queue[head++];
        // TODO 1: Run p for min(quantum, left[p]) time units: print " P(time-newtime)",
        // advance time, reduce left[p]; add every process that has now arrived and is //
        not yet added to the queue; then put p back at the end of the queue if it // still
        has time left, otherwise record finish[p] and count it as done


    }
    double tw = 0, tt = 0;
    printf("\nProcess TAT WT\n");
    for (int i = 0; i < N; i++) {
        int tat = finish[i] - arrival[i], wt = tat - burst[i];
        tt += tat;
        tw += wt;
        printf("%-7s %4d %3d\n", name[i], tat, wt);
    }printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
                                        return
                                    }   0;
```

Expected output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21)
P1(21-23) P3(23-26)
Process TAT WT
P1 23 15
P2 15 11
P3 24 15
P4 18 13
Average TAT = 20.00, average WT = 13.50
```

Run the program with quantum 1 and quantum 20, and evaluate how the average waiting time and the number of context switches change.

Alternative Concept and Complexity Analysis

If the service used a multilevel feedback queue (short quantum for new jobs, longer quanta for jobs that use them up) instead of a single Round Robin queue, what would the solution be? Evaluate the average waiting time for short and long jobs, the scheduling cost per decision (O(1) per queue versus O(n) for SJF) and the risk of starvation.

Evidence Expected

Correct Gantt charts and averages for every algorithm, a working Round Robin program, a quantum study and a justified recommendation.

Q3 – Threads 6 Marks Bloom's Level: K5 – Evaluate

Problem Context – Traffic-Camera Number-Plate Reader

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind during peak hours.

Your Challenge

Process the camera frames with POSIX threads (one thread per camera, and a pool of 4, 8 and 16 threads). Determine:

- Which resources threads share and which each thread has of its own
- The difference between user-level and kernel-level threads and the many-to-one, one-to-one and many-to-many models
- The speed-up obtained on your machine, and why it stops increasing
- What happens to shared counters when two threads update them without protection

Alternative Concept and Complexity Analysis

If the camera frames were processed by several processes instead of threads, what would the solution be? Evaluate the creation and switching time, the memory used (separate address spaces versus shared), and how the results would be combined (IPC versus shared variables).

Evidence Expected

A working multithreaded program, measured speed-ups for each pool size and a reasoned evaluation of the thread models.

Q4 – Inter-Process Communication 6 Marks Bloom's Level: K5 – Evaluate

Problem Context – Weather Station Data Pipeline

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings.

Your Challenge

Implement the exchange using a pipe, and then using shared memory or a message queue. Determine:

- How the pipe signals end of data to the reader
- The difference between the shared-memory and message-passing models •
  Which method copies the data and which does not
- Which method needs extra synchronisation, and why

Growth Investigation

Measure the time to transfer 1 MB, 10 MB and 100 MB with each method, and evaluate the results.

Complete the Code

Complete TODO 1 below and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

/* The parent (sensor reader) sends readings to the child (analyser) through a pipe. */
int main(void)
{int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid == 0) {
        close(fd[1]); /* child only reads */
        int r, count = 0, sum = 0, max = 0;
        // TODO 1: Read one int at a time from fd[0] until read returns 0 (the parent
        // closed the pipe); keep the count, the sum and the maximum

        printf("analyser: %d readings, average %.1f, max %d\n", count,
               count ? (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
                                          _exit(0
                                    }     );

    close(fd[0]); /* parent only writes */ int readings[] = {31, 33,
    36, 34, 41, 38};
    for (int i = 0; i < 6; i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]); /* signals end of data to the child */  waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

Remove close(fd[1]) in the parent and explain why the child never

finishes. **Alternative Concept and Complexity Analysis**

If the readings were exchanged through shared memory with a semaphore instead of a pipe, what would the solution be? Evaluate the time to transfer n readings (two copies through the kernel for a pipe versus none for shared memory) and the synchronisation needed.

Evidence Expected

Two working IPC implementations, measured transfer times and a justified choice of method.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors 11 Marks Bloom's

Level: K5 – Evaluate

Problem Context – Railway Ticket Booking

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with a single global lock the system becomes very slow.

Design Starting Point

You are given only the following principle:

> **A solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.**

Your Challenge

Convert this principle into a design. Determine:

- Why Peterson's solution satisfies the three requirements for two processes, and why it may fail on modern processors
- How counting and binary semaphores (wait and signal) protect the seat counter • How the bounded-buffer (producer–consumer), readers–writers and dining-philosophers problems are solved with semaphores
- How a monitor with condition variables makes the solution easier to get right

Test Your Design

Run the bounded-buffer program with 1 and with 4 producers and consumers, and show that every item is consumed exactly once.

Deployment Decision

Decide between one lock for the whole booking system, one lock per train and one lock per coach. Measure the throughput of each and defend your choice.

Complete the Code

Complete TODO 1 and TODO 2 below using the semaphores and the mutex, and execute the program (compile with -pthread).

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20
```

```
int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;

void *producer(void *arg)
{
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        // TODO 1: Wait on empty, lock the mutex, put item at buffer[in], advance in
        // circularly, unlock, then post full

    }
    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        // TODO 2: Wait on full, lock the mutex, take the item at buffer[out], advance
        // out circularly, unlock, post empty and add the item to consumed_sum

    }return NULL;
}

int main(void)
{sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n", ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);
    sem_destroy(&empty);
    sem_destroy(&full);
                                        return
                                    }   0;
```

Expected output:

```
items consumed: 20, sum = 210 (expected 210)
```

Run the program 100 times and show that the sum is always 210. Then remove the mutex and evaluate whether the result is still correct with several producers and consumers.

Alternative Concept and Complexity Analysis

If the shared counter were protected by atomic operations (C11 atomic_fetch_add) or by a spin lock instead of a semaphore and mutex, what would the solution be? Evaluate the time per update with 1, 4 and 16 threads, the CPU time wasted by spinning, and which method suits a long critical section.

Evidence Expected

A student-derived synchronisation design, working semaphore code, test evidence and a measured, defended locking decision.

DAY-4 DESIGN CHALLENGE

Q6 – Deadlock Handling and the Banker's Algorithm 13 Marks Bloom's Level: K5 – Evaluate

Problem Context – Shared Laboratory Equipment Scheduler

A research centre runs jobs that need several shared resources at once (GPUs, licences for simulation software and high-speed storage). Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes.

A. Characterise Deadlock – 2 Marks

State the four necessary conditions for deadlock and show them in a resource-allocation graph for the cluster.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R] = {3, 3, 2};
int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};

/* Returns 1 if the state is safe and stores a safe sequence in seq. */
int is_safe(int seq[P])
{int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;
            // TODO 1: If need = maxm[p] - alloc[p] is <= work for every resource, let p
            // finish: add alloc[p] to work, mark finish[p], append p to seq and set //
            found = 1


        }if (!found) return 0;
                                              }return
                                      }       1;


void request(int p, const int req[R])
{for (int r = 0; r < R; r++)
         if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
             printf("P%d request: must wait (not available)\n", p);
                                              return;
                                      }
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }
    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    } else {
          for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }
                          printf("P%d request:
}                         denied (unsafe)\n", p);
}



int main(void)
{int seq[P];
    if (is_safe(seq)) {
        printf("initial state safe:");
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    }
    request(1,  (int[R]){1,  0,  2});
    request(4,  (int[R]){3,  3,  0});
    request(0,  (int[R]){0,  2,  0});
    return 0;
}
```

Expected output:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

Show by hand why the request of P0 for (0, 2, 0) leads to an unsafe state.

Alternative Concept and Complexity Analysis

If deadlock were handled by detection and recovery (running a detection algorithm every few minutes and aborting a job) instead of the Banker's algorithm, what would the solution be? Evaluate the time per request ($O(m \times n^2)$ for every Banker's check versus no check), the cost of detection, and the work lost when a job is aborted.

Evidence Expected

A correct resource-allocation graph identifying all four conditions.

B. Compare the Strategies – 3 Marks

Evaluate prevention (removing one condition), avoidance (Banker's algorithm), detection and recovery, and ignoring deadlock (the ostrich approach) for the cluster.

Evidence Expected

A reasoned comparison of the four strategies for this environment.

C. Apply the Banker's Algorithm – 4 Marks

Implement the safety algorithm and the resource-request algorithm. Find a safe sequence for the given state and decide whether each request can be granted.

Evidence Expected

A working implementation with correct safe sequences and decisions.

D. Predict Before Implementing – 1 Mark

Predict how the running time of the Banker's algorithm grows with the number of processes n and resource types m.

Evidence Expected

A clear, testable prediction.

E. Measure – 1 Mark

Measure the time of the safety algorithm for 10, 100 and 1,000 processes and compare it with your prediction.

Evidence Expected

	Measured times compared with the prediction.
F. Defend the Final Design – 2 Marks	Recommend the deadlock strategy for the research centre and justify it, including the cost of the Banker’s algorithm and the need to know maximum demands in advance.
Evidence Expected	A critical, evidence-based recommendation.

Introduction and Experimental Set-up

This report answers the six design challenges in the Operating Systems practical set. For each question it follows the same pattern that the evaluation sheet asks for: a short statement of the problem, the required diagrams and theory, a working program with real output, an evaluation of the results, a defended design decision and an analysis of the alternative concept.
Every program in this report was written in C (or Python for the pencil-and-paper style scheduling tables), compiled and executed. The numbers quoted are measured values, not textbook values. Where a number is an analytical estimate it is labelled as an estimate.

Test machine

Item	Value
Operating system	Linux 6.18 (x86-64), Ubuntu 24.04 user-space
Processor	Intel Xeon @ 2.80 GHz, **1 virtual CPU**
Memory	4 GB RAM
Compiler	gcc 13.3, `-O2 -pthread` (a few demos use `-O1` so the racy code is not optimised away)
Tools	`ps`, `/proc`, `timeout`, `python3`

Important limitation. The machine has a single virtual CPU. Any experiment that depends on true parallel execution (speed-up of CPU-bound threads, lock contention across cores) therefore shows the behaviour of a one-core system. Each such result is explained, and the expected multi-core behaviour is stated separately and marked as an expectation rather than a measurement.

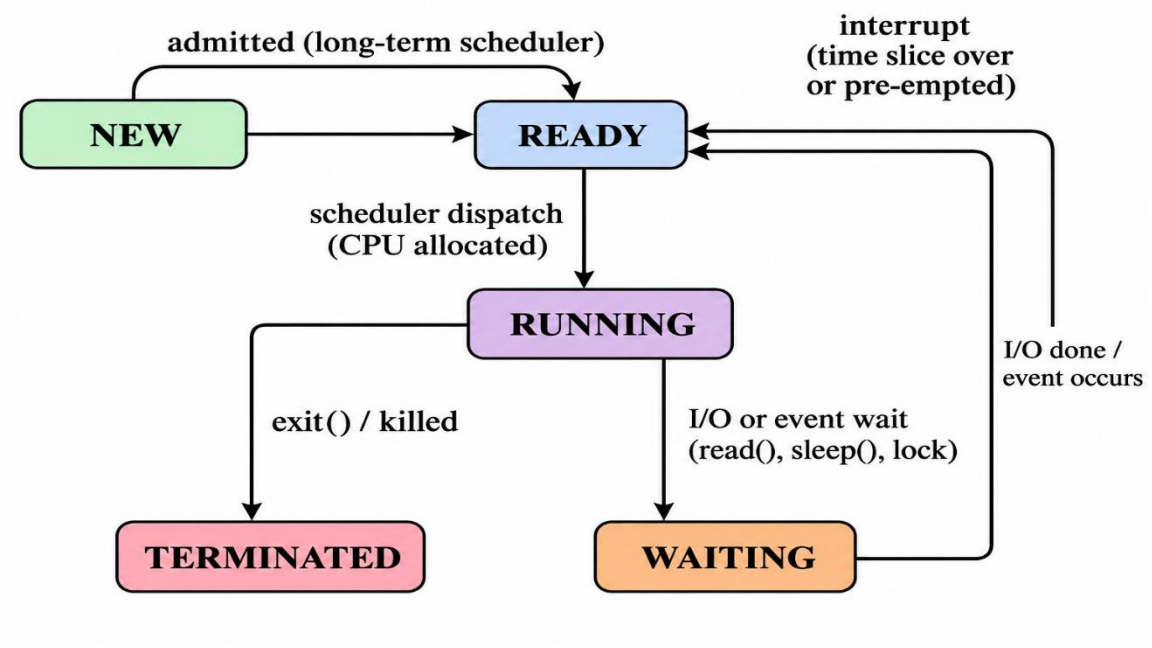
Q1. Processes and Process States

1.1 Problem summary

An online examination server runs one process for every student session, plus helper processes for auto-saving answers and for proctoring video. When 2,000 students log in together the server slows down and some sessions appear to hang. The task is to model processes, observe them with `fork()`, `ps` and `/proc`, evaluate where the time goes and decide on the right concurrency design.

1.2 Process-state diagram

A process is always in exactly one of five states. The diagram below shows the states and the events that cause each transition.



Transition	Cause	Example in the exam server
New to Ready	The OS accepts the process and allocates its PCB and memory	fork() returns in the child; the new session is created
Ready to Running	The short-term scheduler (dispatcher) picks it	A student's session gets the CPU to render the next question
Running to Ready	Time slice expires or a higher-priority task becomes ready	Another session or the proctoring encoder pre-empts it
Running to Waiting	The process requests I/O or waits for an event	Session waits for the database to confirm an answer was saved
Waiting to Ready	The I/O completes (an interrupt) or the event occurs	The disk write finishes; the session becomes runnable again
Running to Terminated	exit(), a fatal signal, or the parent kills it	The student submits and the session ends

In Linux the states appear in ps as R (running or runnable), S (interruptible sleep, which is the Waiting state), D (uninterruptible sleep, usually disk I/O), T (stopped), Z (zombie, which is terminated but not yet reaped by the parent). The 'New' state is very short in Linux because fork() builds the child and puts it straight on the run queue.

1.3 Process control block (PCB)

The PCB is the kernel's record of everything it needs to stop a process and restart it later. In Linux it is the task_struct. The table lists the logical fields and where each one can be seen from user space.

PCB field group	Content	Where to observe it
Identification	PID, parent PID (PPID), process-group and session id, user and group id	ps -o pid,ppid, /proc/PID/status (Pid, PPid, Uid)
Process state	new, ready, running, waiting, terminated	ps -o stat, /proc/PID/status (State)
CPU context	Program counter, stack pointer, general registers, flags	Saved in the kernel stack at every switch (not visible directly)
Scheduling information	Priority, nice value, time slice, scheduling class, run-queue links	ps -o ni,pri, /proc/PID/sched
Memory management	Page-table base (CR3), segment/VMA list, text, heap and stack limits	/proc/PID/maps, VmSize and VmRSS in /proc/PID/status
I/O and file information	Table of open file descriptors, current directory, signal handlers	ls /proc/PID/fd
Accounting	CPU time used, start time, number of context switches	/proc/PID/stat, voluntary_ctxt_switches in /proc/PID/status
Wait information	What the process is blocked on (wait channel)	ps -o wchan

At a context switch the kernel saves the registers of the outgoing process into its PCB, picks the next PCB from the ready queue, loads the next process's registers and page-table base, and resumes it. Switching costs no useful work, so it is pure overhead.

1.4 Program: creating child processes with fork()

The program creates *n* children. Each child prints its identity and then sleeps for four seconds to imitate a student session waiting for I/O. The parent waits one second and then inspects the children with ps and /proc before reaping them with wait().

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char **argv)
{
    int n = argc > 1 ? atoi(argv[1]) : 4;
    pid_t kids[64];
    printf("parent pid=%d\n", getpid());
    for (int i = 0; i < n; i++) {
        pid_t p = fork();
        if (p < 0) { perror("fork"); exit(1); }
        if (p == 0) {                       /* child = one student session
```

```
*/
            printf("child %d: pid=%d ppid=%d\n", i, getpid(), getppid());
            fflush(stdout);
            sleep(4);                       /* pretend to wait for I/O */
            _exit(10 + i);
        }
        kids[i] = p;
    }
    sleep(1);
    char cmd[256];
    snprintf(cmd, sizeof cmd,
      "ps -o pid,ppid,stat,wchan:14,rss,cmd --ppid %d", getpid());
    system(cmd);
    snprintf(cmd, sizeof cmd,
      "grep -E 'Name|State|Pid|PPid|VmSize|VmRSS|ctxt' /proc/%d/status",
kids[0]);
    system(cmd);
    for (int i = 0; i < n; i++) {
        int st;
        pid_t w = wait(&st);
        printf("reaped pid=%d exit=%d\n", w, WEXITSTATUS(st));
    }
    return 0;
}
```

Output of ./q1_fork 4 (abridged to the parts that matter):

```
child 0: pid=114 ppid=113
child 1: pid=115 ppid=113
child 2: pid=116 ppid=113
child 3: pid=117 ppid=113

--- ps (children of 113) ---
  PID  PPID STAT WCHAN            RSS CMD
  114   113 S    hrtimer_nanosl   892 ./q1_fork 4
  115   113 S    hrtimer_nanosl   892 ./q1_fork 4
  116   113 S    hrtimer_nanosl   892 ./q1_fork 4
  117   113 S    hrtimer_nanosl   892 ./q1_fork 4

--- /proc/114/status (first child) ---
Name:   q1_fork
State:  S (sleeping)
Pid:    114
PPid:   113
VmSize:     2692 kB
VmRSS:       892 kB
voluntary_ctxt_switches:    1
nonvoluntary_ctxt_switches: 0

reaped pid=114 exit=10
reaped pid=115 exit=11
reaped pid=116 exit=12
reaped pid=117 exit=13
```

1.5 What the evidence shows

1. **Parent and child relationship.** All four children report ppid=113, the parent's PID, and each has its own PID. This is the process tree that the PCB's identification fields describe.
2. **State observation.** ps shows state S and the wait channel hrtimer_nanosleep: the children are in the *Waiting* state, blocked on a timer, and use no CPU. This is exactly what a student session looks like while it waits for a database reply.

3. **Context-switch counters.** voluntary_ctxt_switches: 1 means the child gave up the CPU once of its own accord (when it called sleep). A *non-voluntary* switch would mean the scheduler forcibly pre-empted it.
4. **Exit status collection.** The parent receives exit codes 10 to 13 through wait(). Until wait() is called a finished child stays in the zombie (Z) state; its PCB cannot be freed. A server that forgets to wait() for 2,000 sessions leaks 2,000 PCBs.
5. **A subtle fork() pitfall that appeared in the raw output.** When standard output is a pipe or file, printf is fully buffered. The line parent pid=113 was still sitting in the stdio buffer when fork() ran, so each child inherited a copy and the line was printed again by every child (it appeared five times in the raw capture). The cure is fflush(stdout) before fork(). The same mechanism is why fork() copies the whole address space: buffers, open files and all.

1.6 Evaluation: where does the time go with 2,000 processes?

Three measurements were made on the test machine, followed by an analysis of the scheduler.
(a) Cost of one context switch. Two processes pass a one-byte message back and forth through a pair of pipes, both pinned to one CPU, so each message forces a switch. 200,000 round trips were timed.

Mechanism	Time per context switch (measured, two runs each)
Two processes	1.26 us and 1.16 us
Two threads of one process	1.09 us and 1.07 us

The figure includes the pipe system calls themselves, so it is an upper bound on the bare switch. The difference between processes and threads is small here because the two tiny processes have almost no memory to reload. In a real server with large working sets the process switch is more expensive: the page-table base register changes, the TLB entries that were tagged for the old address space become useless (unless the CPU supports address-space tags) and the CPU caches are refilled by the new process. The raw switch is only part of the story.
(b) Cost of creating and holding the sessions. A parent with a 20 MB heap (like a real server) created 2,000 sleeping children, and then 2,000 sleeping threads (64 KB stacks) for comparison. The figures are the best of two runs, with the additional memory taken from MemAvailable in /proc/meminfo.

2,000 sessions as	Creation time total	Per session	Extra memory total	Per session
Processes (fork)	313 ms to 394 ms	157 to 197 us	about 300 MB	about 152 KB
Threads (pthread_create)	38 ms to 40 ms	19 to 20 us	32 to 64 MB	16 to 33 KB

The per-process cost is much larger than the 892 KB resident set of a trivial child suggests, because fork() must copy the parent's page tables (the 20 MB heap needs about 5,000 page-table entries), allocate a kernel stack, a task_struct and a new address-space descriptor.

Memory itself is shared copy-on-write, but the bookkeeping is not. The larger and busier the parent, the more expensive each fork becomes.

(c) Time spent waiting. A student session spends most of its life waiting for the network, the database or the student. In the `ps` output the children are in `S` with a wait channel; they consume no CPU. Waiting itself is harmless. The damage comes from the few processes that are *runnable at the same moment*: the proctoring video encoders and the auto-save bursts.

(d) Scheduling latency analysis (estimate, not measured). In Linux's fair scheduler every runnable task should run once in each scheduling period. With *R* runnable tasks and a minimum slice of about 0.75 ms, a task can wait up to R x 0.75 ms between turns. If 2,000 sessions become runnable together (everyone clicks 'Next' after the same announcement), one full rotation takes roughly 2,000 x 0.75 ms = 1.5 s, and any session that is also waiting on a lock, a disk or a swapped-out page will look frozen. Adding 2,000 video-encoding processes on top makes memory pressure worse: if resident memory exceeds RAM the system pages to disk, a process waits in `D` state for each page fault, and the whole server crawls. This is the likely cause of the 'hanging' sessions.

Summary of the cost breakdown for 2,000 processes

Cost	Size	Comment
Context switches	about 1 us each raw, more with cache and TLB refill	Grows with the number of runnable processes, not with the total
Memory	about 300 MB for kernel and page-table bookkeeping alone, before any application data	Full private data per session adds much more
Creation	0.3 to 0.4 s for the burst	Visible as a login storm delay
I/O waiting	Dominant fraction of each session's life	Free if the CPU is shared properly, fatal if memory is exhausted

Judgement. A separate process per student is *not* a sound design for this workload. It pays full isolation cost (page tables, kernel structures, a fork per login) for sessions that are small and mostly idle, it has no upper bound on resource use, and it gives the scheduler thousands of entities to rotate through. Isolation between students is not needed at address-space level, because the sessions run the same trusted code.

1.7 Design decision

Criterion	One process per student	One thread per student	Fixed pool of worker processes
Creation cost	Highest (about 160 to 200 us each)	Low (about 20 us each)	None per session (workers exist already)
Memory per session	About 150 KB plus data	About 16 to 33 KB plus data	Only the session data

Criterion	One process per student	One thread per student	Fixed pool of worker processes
Upper bound on resources	None: 2,000 logins means 2,000 processes	None: 2,000 logins means 2,000 threads	Fixed by design: the pool size
Scheduler pressure	Thousands of entities	Thousands of entities	About one worker per CPU core
Fault isolation	Perfect: one crash kills one session	Poor: one crash kills every session	Good: one crash loses only that worker's sessions, and the supervisor restarts it
Programming effort	Easiest	Needs locks for shared data	Needs an event loop or queue and state kept outside the worker
Behaviour under overload	Collapses	Degrades badly	Queues requests and keeps latency predictable

Decision: a fixed pool of worker processes, sized at roughly the number of CPU cores (plus a few for blocking work), with each worker serving many sessions using non-blocking I/O or a small internal thread pool. The CPU-heavy proctoring video encoding runs in a separate, smaller pool so that it can never starve the answer-saving path. A supervisor process restarts any worker that dies.

Defence. The requirement is to survive a login storm without hanging. A pool is the only option of the three that puts a ceiling on memory, process count and run-queue length: excess requests wait in a queue (a controlled delay) instead of becoming thousands of competing processes (an uncontrolled slowdown). It keeps process-level isolation between groups of students, which a pure thread design loses, and it removes the per-login `fork()` cost measured above. Its price is more design effort: session state must be kept in a shared store (a database or an in-memory cache) so that any worker can serve any student and a restarted worker can resume sessions.

1.8 Alternative concept and complexity analysis: one thread per session inside one server process

Solution. The server starts as a single process. A listener thread accepts each login and creates a thread for it (`pthread_create`). The session threads share the heap, code and open files, so exam data structures (question bank, answer cache) exist once. Each thread has only its own stack, registers and thread-local storage. Auto-save and proctoring become further threads, or a small pool of threads.

Context-switch time. Switching between threads of the same process keeps the same page tables, so the TLB and the cache of the shared data stay valid. Measured: about 1.07 to 1.09 us for threads against 1.16 to 1.26 us for processes, about 8 to 15 percent lower even in the best case for processes. With real working sets the gap widens.

Memory per session. A few kilobytes of stack and thread descriptor against a full address space descriptor with page tables. Measured: 16 to 33 KB per thread (with 64 KB stack reservations; most of it never touched) against about 152 KB per process. For 2,000 sessions that is roughly 32 to 64 MB against 300 MB. Thread creation is about eight to ten times faster (20 us against 157 to 197 us).
Effect of one session crashing. This is the weakness. A segmentation fault in any thread terminates the *whole process*, because the signal action is process-wide. One bad session can therefore end all 2,000 exams. A memory-corrupting bug in one thread can also silently damage another student's data because everything is shared. The design also needs locks around shared structures, which brings the race-condition risks examined in Q3 and Q5.
Conclusion of the comparison. Threads fix the memory and creation cost but worsen fault isolation. The recommended compromise is therefore the hybrid in the design decision: a *small pool of processes*, each containing *threads or an event loop* for its share of the students.

1.9 Evidence checklist for Q1

Evidence expected	Where it is provided
State diagram	Section 1.2
PCB description	Section 1.3
Working fork() program with ps evidence	Sections 1.4 and 1.5
Evaluation of context-switch cost, memory per process and I/O waiting	Section 1.6
Defended design decision	Section 1.7
Alternative thread-based concept with complexity analysis	Section 1.8

Q2. CPU Scheduling

2.1 Problem summary

A cloud video-encoding service runs short jobs (thumbnails, about 2 seconds) and long jobs (full films, about 20 minutes) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used. The task is to compute Gantt charts and the waiting, turnaround and response times for five algorithms, implement Round Robin, study the time quantum, discuss starvation and ageing, recommend a policy and analyse a multilevel feedback queue.

2.2 Definitions used

Measure	Formula	Meaning
Completion time (CT)	Time at which the job finishes	
Turnaround time (TAT)	CT minus arrival time	Total time in the system
Waiting time (WT)	TAT minus burst time	Time spent in the ready queue
Response time (RT)	First time on the CPU minus arrival time	How long until the user sees any progress
Context switch	The CPU moves from one process to a different one	Pure overhead

2.3 Job set for the hand calculation

The job set is the one embedded in the Round Robin program supplied with the question. The question does not give priorities, so for the priority algorithm they are *assumed* (a smaller number means a higher priority, and the algorithm is non-preemptive).

Process	Arrival	Burst	Priority (assumed)
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2

The supplied results table (P1: TAT 23, WT 15; P2: 15 and 11; P3: 24 and 15; P4: 18 and 13; averages 20.00 and 13.50) belongs to Round Robin with quantum 3. It is reproduced exactly by the program in section 2.5, which confirms that the job set and the algorithm are the ones intended.

2.4 Results for the five algorithms

All tables below were produced by a scheduling simulator written for this report (Python) and were checked by hand for the first three algorithms.

FCFS (first come, first served)

Jobs run in order of arrival, to completion.

Gantt: P1(0-8) P2(8-12) P3(12-21) P4(21-26)

Process	CT	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7

Process	CT	TAT	WT	RT
P3	21	19	10	10
P4	26	23	18	18
Average		**15.25**	**8.75**	**8.75**

Hand check for P4: it arrives at 3 and starts at 21, so WT = 21 - 3 = 18, TAT = 26 - 3 = 23.

SJF (shortest job first, non-preemptive)

At time 8 the ready jobs are P2 (4), P3 (9) and P4 (5), so P2 is chosen. At time 12 the choice is P4 (5) over P3 (9).

Gantt: P1(0-8) P2(8-12) P4(12-17) P3(17-26)

Process	CT	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7
P3	26	24	15	15
P4	17	14	9	9
Average		**14.25**	**7.75**	**7.75**

SRTF (shortest remaining time first, preemptive SJF)

At time 1 P2 (burst 4) arrives and is shorter than P1's remaining 7, so it pre-empts P1. P4 (5) arrives at 3 but P2 has only 2 left, so P2 continues. At time 5 P4 (5) beats P1 (7) and P3 (9).

Gantt: P1(0-1) P2(1-5) P4(5-10) P1(10-17) P3(17-26)

Process	CT	TAT	WT	RT
P1	17	17	9	0
P2	5	4	0	0
P3	26	24	15	15
P4	10	7	2	2
Average		**13.00**	**6.50**	**4.25**

Priority scheduling (non-preemptive, assumed priorities)

After P1 finishes at 8 the highest priority (smallest number) among P2 (1), P3 (4) and P4 (2) is P2, then P4, then P3. With these priorities the order coincides with SJF.

Gantt: P1(0-8) P2(8-12) P4(12-17) P3(17-26)

Process	CT	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7

Process	CT	TAT	WT	RT
P3	26	24	15	15
P4	17	14	9	9
Average		**14.25**	**7.75**	**7.75**

Round Robin, quantum 3

Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26)

Process	CT	TAT	WT	RT
P1	23	23	15	0
P2	16	15	11	2
P3	26	24	15	4
P4	21	18	13	6
Average		**20.00**	**13.50**	**3.00**

Side-by-side summary

Algorithm	Avg TAT	Avg WT	Avg RT	Context switches	Comment
FCFS	15.25	8.75	8.75	3	Simple, convoy effect
SJF (non-preemptive)	14.25	7.75	7.75	3	Needs burst estimates
SRTF	**13.00**	**6.50**	4.25	4	Optimal average waiting time
Priority (non-preemptive)	14.25	7.75	7.75	3	Depends entirely on priorities given
Round Robin q = 3	20.00	13.50	**3.00**	9	Best response, worst turnaround

SRTF has the smallest average waiting and turnaround time, as the theory predicts. Round Robin has the best *response* time among the algorithms that do not need burst knowledge but the worst waiting and turnaround, because every job is repeatedly interrupted and finishes late.

2.5 Implementation: Round Robin

The body of the loop runs the process at the head of the queue for min(quantum, left) time units, advances the clock, enqueues every process that has arrived in the meantime, and only then puts the interrupted process at the back. Adding the new arrivals *before* the pre-empted process is the standard convention and the one that reproduces the expected output. Two extra

counters were added: the quantum can be given on the command line, and context switches (a different process following another) are counted.

```
#include <stdio.h>
#include <stdlib.h>
#define N 4
int main(int argc, char **argv)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int quantum = argc > 1 ? atoi(argv[1]) : 3;
    int left[N], finish[N], queue[256], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];
    int time = 0, done = 0, switches = 0, last = -1;
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while (done < N) {
        int p = queue[head++];
        /* TODO 1 (completed) */
        int run = left[p] < quantum ? left[p] : quantum;
        printf(" %s(%d-%d)", name[p], time, time + run);
        if (last != -1 && last != p) switches++;
        last = p;
        time += run;
        left[p] -= run;
        for (int i = 0; i < N; i++)
            if (!added[i] && arrival[i] <= time) { queue[tail++] = i;
added[i] = 1; }
        if (left[p] > 0) queue[tail++] = p;
        else { finish[p] = time; done++; }
    }
    double tw = 0, tt = 0;
    printf("\nProcess TAT WT\n");
    for (int i = 0; i < N; i++) {
        int tat = finish[i] - arrival[i], wt = tat - burst[i];
        tt += tat; tw += wt;
        printf("%-7s %4d %3d\n", name[i], tat, wt);
    }
    printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
    printf("Context switches = %d\n", switches);
    return 0;
}
```

Output with quantum 3 (identical to the expected output and to the results table in the question):

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26)
Process  TAT  WT
P1       23   15
P2       15   11
P3       24   15
P4       18   13
Average TAT = 20.00, average WT = 13.50
Context switches = 9
```

2.6 Time-quantum study (quantum 1, 3, 5 and 20)

The program was run with each quantum.

quantum 1 : Gantt: P1(0-1) P2(1-2) P1(2-3) P3(3-4) P2(4-5) P4(5-6) P1(6-7) P3(7-8) P2(8-9) P4(9-10)
P1(10-11) P3(11-12) P2(12-13) P4(13-14) P1(14-15) P3(15-16) P4(16-17) P1(17-18) P3(18-19)
P4(19-20) P1(20-21) P3(21-22) P1(22-23) P3(23-24) P3(24-25) P3(25-26)
TAT/WT: P1 23/15 P2 12/8 P3 24/15 P4 17/12 avg TAT 19.00, avg WT 12.50
quantum 5 : Gantt: P1(0-5) P2(5-9) P3(9-14) P4(14-19) P1(19-22) P3(22-26)
TAT/WT: P1 22/14 P2 8/4 P3 24/15 P4 16/11 avg TAT 17.50, avg WT 11.00
quantum 20: Gantt: P1(0-8) P2(8-12) P3(12-21) P4(21-26)
TAT/WT: P1 8/0 P2 11/7 P3 19/10 P4 23/18 avg TAT 15.25, avg WT 8.75

Quantum	Avg WT	Avg TAT	Avg RT	Context switches	Dispatches
1	12.50	19.00	**0.75**	23	26
3	13.50	20.00	3.00	9	10
5	11.00	17.50	5.50	5	6
20	**8.75**	**15.25**	8.75	**3**	4

Evaluation.

- **Quantum 1** gives the best response time (0.75 time units: every job gets the CPU almost immediately) but needs 23 context switches for only 26 units of work. If a switch costs even 5 percent of a quantum, the effective throughput falls by about the same amount. The measured raw switch cost on this machine was about 1.2 microseconds (Q1), which is negligible against a 1-millisecond quantum but becomes significant once cache and TLB refills are included, and it is a heavy load for a 1-unit quantum on real hardware.
- **Quantum 20** is larger than every burst, so no job is ever pre-empted and Round Robin *degenerates into FCFS*: the Gantt chart, the averages (8.75 and 15.25) and the 3 context switches are identical to FCFS. It has the fewest switches but the worst response time (8.75) and brings back the convoy effect.
- **Quantum 3 and 5** lie in between. The averages are not perfectly monotonic (q = 3 gave a higher average waiting time than q = 1 and q = 5) because with only four jobs the order in which the last bursts fall inside a quantum matters. The reliable trend is on the *response* time (rises with the quantum) and on the *switch count* (falls with the quantum).
- **Rule of thumb** from this study: choose a quantum a little larger than the typical short burst, so that most short jobs finish within one quantum while long jobs are still rotated. A common guideline is that about 80 percent of CPU bursts should be shorter than the quantum.

2.7 Starvation, ageing and multilevel feedback queues

Starvation in SJF and priority scheduling. Both algorithms always pick the 'best' job by some key (shortest burst, highest priority). If better jobs keep arriving, a worse job is never picked. In the video service the thumbnails (2 s) would indefinitely postpone a film under SJF

or SRTF, and a low-priority batch encode would never run under a strict priority policy while interactive jobs keep coming.

Measured demonstration. A low-priority job *L* (burst 10, priority 9) arrives at time 2, while a stream of 40 high-priority jobs (burst 5, priority 1) arrives every 4 time units. Without ageing it must wait for all of them; with ageing the priority of a waiting job improves by 1 for every *k* time units it waits.

Policy	L starts at	Waiting time of L	L finishes at
Priority, no ageing	200	198	210 (after all 40 high-priority jobs)
Priority with ageing, 1 level per 10 units	95	93	105
Priority with ageing, 1 level per 5 units	55	53	65

Ageing gradually raises the priority of a job in proportion to its waiting time. After a bounded time even the lowest-priority job outranks the newcomers, so its waiting time has an upper bound and starvation is impossible. The ageing rate trades fairness against the priority guarantees of the high-priority class.

Multilevel feedback queue (MLFQ). The scheduler keeps several queues, from highest to lowest priority. The standard rules are:

1. A new job enters the top queue, which has the shortest quantum.
2. If the job uses its whole quantum without finishing or blocking, it is demoted one level (longer quantum, lower priority).
3. If it finishes or blocks before the quantum ends, it stays at its level (interactive and short jobs are rewarded).
4. The CPU always serves the highest non-empty queue, and a newly arrived job in a higher queue pre-empts a lower one.
5. Starvation control: a job that waits too long is promoted (ageing), or all jobs are periodically boosted to the top queue.

A **short job** (a thumbnail) finishes inside the first, short quantum and is never demoted, so it leaves almost immediately. A **long job** (a film) uses up its quantum at every level and sinks to the bottom queue, where it receives long quanta (fewer switches, better throughput) but only when no shorter job is waiting. The scheduler needs no prior knowledge of the burst length; it *learns* it from the behaviour of the job.

2.8 Design decision: the scheduling policy for the video service

To judge the policies on the real workload, a second experiment used a model of the service: three films of 1,200 seconds arriving at times 0, 10 and 20, and 20 thumbnails of 2 seconds arriving every 5 seconds from time 1. Total work is 3,640 seconds, and it is the same for every policy, so a work-conserving policy keeps the processor 100 percent busy until the queue is empty.

Policy	Short jobs: avg WT (s)	Short jobs: avg TAT (s)	Long jobs: avg WT (s)	Long jobs: avg TAT (s)	Overall avg WT (s)	Context switches
FCFS	3,210.50	3,212.50	1,194.00	2,394.00	2,947.48	22
SJF (non-preemptive)	1,170.50	1,172.50	1,216.67	2,416.67	1,176.52	22
SRTF	**0.00**	**2.00**	1,230.00	2,430.00	**160.43**	42
Round Robin, q = 10 s	33.00	35.00	2,416.67	3,616.67	343.91	379
Round Robin, q = 100 s	240.50	242.50	2,330.00	3,530.00	513.04	55
Multilevel feedback, quanta 4, 16, 64 s	0.45	2.45	2,419.67	3,619.67	316.00	98

What the numbers say.

- **FCFS is the cause of the complaints.** A 2-second thumbnail waits on average 3,210 seconds (about 53 minutes) behind the films. This is the convoy effect.
- **Non-preemptive SJF is not enough.** It only chooses between waiting jobs; a thumbnail that arrives while a film is running still waits for the film to end (1,170 s on average).
- **SRTF gives the best averages** (thumbnails wait 0 s, overall wait 160 s) but needs to know each job's remaining time in advance. For video, the length of a file may be known, but the encoding time depends on the codec, content and hardware, and the estimates would have to be maintained.
- **Round Robin** protects the short jobs only if the quantum is small: q = 10 gives 33 s of waiting for thumbnails but costs 379 context switches and makes the films finish much later; q = 100 reduces the switch count to 55 but short jobs wait 240 s again.
- **The multilevel feedback queue** gives thumbnails almost the same service as SRTF (0.45 s average wait) without any prior knowledge of job lengths, with only 98 switches.

Recommendation: a multilevel feedback queue with three or four levels (short quanta at the top), ageing or a periodic priority boost, and *first-come first-served in the lowest queue* so that the long films complete one after another instead of all being delayed together.

The measured table shows the cost honestly: the films' average turnaround rises from 2,394 s (FCFS) to 3,620 s (MLFQ in the experiment) because the thumbnails take about 40 percent of the CPU in the stress workload, and because the simulated bottom level used Round Robin among the three films, which makes them finish together. Using FCFS at the bottom level should recover most of that penalty (expected from the policy's logic, not measured here). The trade is acceptable because the customers who complain are the ones waiting 53 minutes for a 2-second job, whereas a film that takes 20 minutes of CPU already has a long, understood delay.

2.9 Alternative concept and complexity analysis: multilevel feedback queue instead of one Round Robin queue

Solution. Replace the single Round Robin queue by three queues Q0 (quantum 4 s), Q1 (quantum 16 s) and Q2 (quantum 64 s, or FCFS), with the rules of section 2.7. This was implemented in the simulator and gives the MLFQ row of the table above.

Average waiting time for short and long jobs (workload of section 2.8):

Scheduler	Short jobs avg WT	Long jobs avg WT
Single Round Robin, q = 10	33.00 s	2,416.67 s
Single Round Robin, q = 100	240.50 s	2,330.00 s
Multilevel feedback	**0.45 s**	2,419.67 s

Short jobs wait 70 to 500 times less than with a single Round Robin queue, while long jobs wait about the same as with q = 10. The MLFQ therefore gets the response of a small quantum for short jobs and the efficiency of a large quantum for long jobs.

Scheduling cost per decision.

Policy	Cost of choosing the next job	Why
Round Robin	O(1)	Take the head of one FIFO queue
Multilevel feedback queue	O(1) per queue, so O(k) for k levels (k is a small constant such as 3)	Take the head of the highest non-empty queue
SJF / SRTF with an unsorted list	O(n)	Scan all n ready jobs for the minimum
SJF / SRTF with a heap	O(log n)	But a new job arriving can force a re-evaluation (SRTF)

On a server with thousands of ready jobs an O(n) scan at every scheduling event is an unacceptable overhead, which is another practical argument for the MLFQ.

Risk of starvation. In the MLFQ a long job is starved when short jobs keep arriving, because the top queues are always non-empty. The mitigations are an ageing rule (promote a job after it has waited for T seconds) or a periodic boost that moves every job back to the top queue every S seconds. The boost also repairs a second weakness: a job that changes from long to short behaviour (for example after finishing a heavy phase) would otherwise stay at the bottom. A user could also *game* the scheduler by issuing a tiny I/O request just before the quantum ends to avoid demotion; accounting for the total CPU time used at each level (rather than the length of each burst) prevents this.

2.10 Evidence checklist for Q2

Evidence expected	Where it is provided
Correct Gantt charts and averages for every algorithm	Section 2.4
Working Round Robin program	Section 2.5 (output equal to the expected output)
Quantum study with 1, 3, 5, 20	Section 2.6
Starvation and ageing evaluated	Section 2.7
Justified recommendation using calculated averages	Section 2.8
Multilevel feedback queue analysis	Section 2.9

Q3. Threads

3.1 Problem summary

A city traffic system reads number plates from 64 cameras. The first version handles the cameras one after another and falls behind at peak hours. The task is to process the frames with POSIX threads (one thread per camera, and pools of 4, 8 and 16 threads) and to determine what threads share, how thread models differ, what speed-up is obtained and why it stops, and what happens to shared counters without protection.

3.2 What threads share and what each thread owns

Shared by all threads of a process	Private to each thread
Program code (text segment)	Thread ID
Global and static variables (data and BSS)	Program counter and CPU registers
The heap (memory from `malloc`)	Its own stack (local variables, return addresses)
Open files and sockets (file-descriptor table)	Thread-local storage (`__thread`, `pthread_key`)
Signal handlers and process ID	Signal mask and `errno`
Current directory, user and group IDs	Scheduling priority and state

Because the heap and globals are shared, communication between threads is just reading and writing memory, which is fast but dangerous without synchronisation (section 3.6). Because each thread has its own stack, local variables are automatically safe.

3.3 User-level and kernel-level threads, and the mapping models

Property	User-level threads	Kernel-level threads
Who manages them	A thread library in user space	The operating-system kernel
Creation and switching	Very fast (no system call)	Slower (a system call and kernel scheduling), but still far cheaper than processes
Kernel awareness	The kernel sees one process	The kernel schedules each thread separately
Blocking system call	Blocks the *whole process*	Blocks only that thread
Multi-core parallelism	None (one kernel entity)	Yes

Mapping model	Description	Strengths	Weaknesses
Many-to-one	Many user threads on one kernel thread	Cheap, portable	No parallelism; one blocking call stops everything
One-to-one	Each user thread has its own kernel thread	True parallelism; blocking is local	Each thread costs kernel resources, so there is a practical limit
Many-to-many	M user threads multiplexed over N kernel threads (N is at most M)	Parallelism without one kernel thread per user thread	Complex; user and kernel schedulers must cooperate

Linux (the native POSIX thread library) and Windows use the **one-to-one** model. It is the model measured in this report. For 64 cameras it is entirely adequate; a many-to-many model would only matter if the system had tens of thousands of threads.

3.4 Program

Each camera job simulates a real plate reader: it first *waits* 10 ms for the frame to arrive from the camera (`usleep`, which is I/O wait) and then does about 9 ms of CPU work (a 4-million-step xorshift loop standing in for image analysis). Four variants are timed: sequential, one thread per camera, and a pool of 4, 8 and 16 worker threads that take the next camera number from a shared atomic counter.

```
#define CAMERAS 64
#define IO_US   10000          /* waiting for the frame */
#define WORK    4000000L       /* plate recognition (CPU) */
static uint64_t process_camera(int c){
    usleep(IO_US);
    uint64_t x = 88172645463325252ULL + c;
    for (long i = 0; i < WORK; i++) { x ^= x << 13; x ^= x >> 7; x ^= x <<
17; }
```

```
    return x & 0xffff;
}
static atomic_int next_cam;
static void *pool_worker(void *a){          /* worker of a pool */
    int c;
    while ((c = atomic_fetch_add(&next_cam, 1)) < CAMERAS)
        result[c] = process_camera(c);      /* each thread writes a
different slot */
    return NULL;
}
static void *one(void *a){                  /* thread-per-camera */
    int c = (int)(intptr_t)a;
    result[c] = process_camera(c);
    return NULL;
}
static double run_pool(int n){
    pthread_t t[CAMERAS]; atomic_store(&next_cam, 0); double t0 = now();
    for (int i = 0; i < n; i++) pthread_create(&t[i], NULL, pool_worker,
NULL);
    for (int i = 0; i < n; i++) pthread_join(t[i], NULL);
    return now() - t0;
}
```

The result array needs no lock because every camera writes only its own element. The shared counter `next_cam` is handled with an atomic operation, so two workers can never take the same camera.

3.5 Measured speed-up

One run on the 1-CPU test machine (the numbers repeat closely from run to run):

```
sequential (1 thread)       : 1.233 s  speed-up 1.00
thread per camera (64)      : 0.577 s  speed-up 2.14
pool of  4 threads          : 0.636 s  speed-up 1.94
pool of  8 threads          : 0.580 s  speed-up 2.13
pool of 16 threads          : 0.572 s  speed-up 2.16
4 worker processes + pipe   : 0.720 s  speed-up 1.71 (64 results)
```

Version	Time (s)	Speed-up
Sequential	1.233	1.00
Pool of 4 threads	0.636	1.94
Pool of 8 threads	0.580	2.13
Pool of 16 threads	0.572	2.16
One thread per camera (64 threads)	0.577	2.14

Why the speed-up stops increasing. One camera takes 1.233 / 64 = 19.3 ms sequentially: about 10 ms of waiting and about 9 ms of computing. With threads, the *waiting* of different cameras overlaps, but the *computing* still has to share the single CPU. The best possible time is therefore about 64 x 9.3 ms = 0.6 s, which is a speed-up of 19.3 / 9.3 = 2.07. The measurements (2.13 to 2.16) are at that ceiling. A pool of 4 is slightly below the ceiling because with only four threads the CPU sometimes sits idle while all four wait for frames;

from 8 threads onwards the CPU is always busy and more threads add nothing (and 64 threads cost more memory for the same result).
General reasons for a speed-up ceiling:

- **Amdahl's law:** speed-up is at most 1 / (s + (1 - s) / p), where s is the serial fraction and p the number of processors. Here the 'serial' resource is the single CPU.
- **Number of cores.** CPU-bound threads beyond the number of cores only time-share them.
- **Sizing rule for a pool:** threads = cores x (1 + wait time / compute time). On this machine: 1 x (1 + 10 / 9.3) = about 2.1, which agrees with the saturation seen at two to eight threads.
- **Overheads:** thread creation, scheduling, lock contention and cache or memory-bandwidth limits, which grow with the thread count.

Expectation for a multi-core server (not measured here): on an 8-core machine the same code should reach a ceiling of roughly 8 x 2.07, limited by the number of cameras (64), memory bandwidth and the camera network, because the compute phase now runs in parallel too.

3.6 Shared counters updated without protection

The statement `counter++` is not one machine instruction. It is a load, an add and a store. Two threads can interleave these steps and lose an update:

Step	Thread A	Thread B	Memory value
1	load counter (5)		5
2		load counter (5)	5
3	add 1 (6)	add 1 (6)	5
4	store 6		6
5		store 6	**6, but two increments were done**

This is a **race condition**: the result depends on the timing. Experiment 1: four threads, each incrementing a shared counter 2,000,000 times, with three kinds of protection.

Method	Final value (expected 8,000,000)	Time
No protection (volatile long)	**2,880,882** (5,119,118 updates lost, 64 percent)	0.018 s
Mutex around each increment	8,000,000	0.195 s
Atomic (atomic_fetch_add)	8,000,000	0.051 s

Experiment 2: two threads, 5,000,000 increments each, three runs of the unprotected program gave 8,264,392, 5,000,000 and 6,774,176 out of 10,000,000. The answer is wrong *and different every time*, which makes such bugs hard to find. Correctness needs a critical section (mutex) or an indivisible hardware operation (atomic). The atomic version is about four times faster than the mutex for this tiny critical section; the unprotected version is fast only because it is wrong.

In the number-plate system the same bug would appear in a counter such as 'plates read today' or in a shared list of detected plates: totals would silently be too low and entries could be overwritten.

3.7 Evaluation of the thread models for this system

The workload mixes waiting and computing and has 64 independent jobs. A one-to-one kernel-thread pool is the natural choice: a thread blocked on a camera does not stop the others, and the compute phases run in parallel on multiple cores. A many-to-one library would serialise the compute phases and a single blocking read would freeze all cameras. A many-to-many model would add complexity without a benefit at this scale.
Recommended design: a pool of kernel threads sized by the rule above (cores x (1 + wait/compute)), with a shared work counter or queue, results written to private slots, and mutex or atomic protection for any shared total.

3.8 Alternative concept and complexity analysis: several processes instead of threads

Solution. Create four worker processes with fork(). Process k handles cameras k, k + 4, k + 8, and so on, and sends each result to the parent through a pipe. The parent reads until end-of-file and then reaps the children. This was implemented and timed above (the last line of the speed-up listing).
Measured and compared with the pool of 4 threads:

Property	4 worker processes + pipe	Pool of 4 threads
Total time for 64 cameras	0.720 s (speed-up 1.71)	0.636 s (speed-up 1.94)
Creation time (Q1 measurement)	about 157 to 197 us each	about 19 to 20 us each
Context switch (Q1 measurement)	about 1.16 to 1.26 us	about 1.07 to 1.09 us
Memory	Separate address space and page tables per process, about 152 KB of kernel bookkeeping each plus private copies of any written data	Shared address space, about 16 to 33 KB per thread
Combining results	IPC: pipe, shared memory or message queue; data is copied through the kernel	Direct reads of shared variables; no copy
Effect of a crash in one worker	Only that worker dies; the parent can restart it	The whole process dies
Synchronisation needed	Almost none for private data; the pipe synchronises itself	Mutexes or atomics for every shared structure

The process version was about 13 percent slower in this test, for three reasons: creating processes and the pipe, an extra copy of every result, and a context switch per pipe handoff. For 64 eight-byte results the copying is negligible; if the workers had to return whole frames of several megabytes, the pipe throughput measured in Q4 (about 5 GB/s) would become a real cost and shared memory would be preferable.

Judgement. Threads win on speed and simplicity of sharing; processes win on fault isolation and on freedom from data races. For a safety-relevant traffic system a hybrid is sensible: a few worker *processes* (so that a crash in the plate-recognition library does not stop the system) each running a *small thread pool* for its share of cameras.

3.9 Evidence checklist for Q3

Evidence expected	Where it is provided
Working multithreaded program	Section 3.4
Measured speed-up for each pool size	Section 3.5
Reasoned evaluation of thread models	Sections 3.3 and 3.7
Shared-counter race demonstrated	Section 3.6
Alternative with processes analysed	Section 3.8

Q4. Inter-Process Communication

4.1 Problem summary

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings. The task is to implement the exchange with a pipe, then with shared memory (and a message queue), to explain how end of data is signalled, to compare the models, and to measure the time to transfer 1 MB, 10 MB and 100 MB.

4.2 Pipe implementation (completing TODO 1)

The parent (sensor reader) writes six readings into the pipe. The child (analyser) reads one integer at a time until `read()` returns 0, counting the readings, summing them and tracking the maximum.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid == 0) {
```

```
        close(fd[1]);                          /* child only reads */
        int r, count = 0, sum = 0, max = 0;
        while (read(fd[0], &r, sizeof r) == sizeof r) {   /* TODO 1 */
            count++;
            sum += r;
            if (r > max) max = r;
        }
        printf("analyser: %d readings, average %.1f, max %d\n", count,
               count ? (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
        _exit(0);
    }
    close(fd[0]);                              /* parent only writes */
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0)
perror("write");
    close(fd[1]);                              /* signals end of data to the
child */
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

Output (identical to the expected output; the average is (31 + 33 + 36 + 34 + 41 + 38) / 6 = 213 / 6 = 35.5):

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

The loop condition compares the return value with `sizeof r` and not just with 0, so a partial read (fewer than four bytes) is also treated as the end of data instead of being counted as a reading.

4.3 How the pipe signals end of data, and what happens if `close(fd[1])` is removed

A pipe has a read end and a write end. `read()` on an empty pipe *blocks* while at least one write end is still open anywhere in the system. It returns 0 (end-of-file) only when the *last* write end has been closed and the pipe is empty. The end of data is therefore signalled implicitly, by closing the write end. No special marker value is needed.

Experiment. The parent's `close(fd[1])` was removed (compiled with `-DNO_CLOSE`) and the program was run under `timeout 3`:

```
--- without close(fd[1]) in parent (timeout 3 s):
exit status 124 (124 = killed by timeout, i.e. hung)
```

Nothing was printed and the program had to be killed. **Explanation:** the parent still holds the write end open while it waits in `waitpid()` for the child to finish. The child reads the six readings and then calls `read()` again; since a write end is still open, `read()` blocks, waiting for data that will never come. The parent waits for the child and the child waits for the parent: a **deadlock**. (The child closed its own copy of the write end at the start; a child that forgot to do so would hang itself even when the parent closed its copy.) The rule is: every process must close the pipe ends it does not use.

The pipe also gives the weather station its 'no lost readings' property. If the analyser is slow and the pipe buffer (64 KB on Linux) is full, `write()` blocks the sensor reader instead of dropping data (back-pressure).

4.4 Shared memory versus message passing

Aspect	Shared memory	Message passing (pipe, message queue)
Principle	Both processes map the same physical pages into their address spaces and read and write them directly	Data is sent through the kernel with send/receive (or write/read) operations
Kernel involvement	Only to set up the mapping; afterwards none	On every message
Copies of the data	**None** after set-up (the producer writes where the consumer reads)	**Two:** user buffer to kernel buffer on write, kernel buffer to user buffer on read
Synchronisation	**Must be added by the programmer** (semaphores, mutexes, flags), otherwise data races occur	**Built in:** the kernel blocks the writer when full and the reader when empty
Message boundaries	None, only bytes; the program must define a layout	Message queues keep boundaries and types; pipes are a plain byte stream
Ease of use	Harder (races, layout)	Easier
Suitability	Large or frequent data on one machine	Small messages; also extends to networks (sockets)

Which method copies the data? The pipe and the message queue copy it twice through the kernel; shared memory does not copy it at all in the transfer itself. (In the benchmark below the producer still performs one `memcpy` from its source buffer into the shared slot, because the benchmark data comes from an ordinary buffer. A sensor driver writing directly into the shared slot would copy nothing.)

Which method needs extra synchronisation, and why? Shared memory. Two processes touching the same memory at the same time without coordination give the lost-update problems of section 3.6, and the consumer could read a slot that the producer has only half written. A pair of semaphores (*empty* slots and *full* slots) prevents both. The pipe synchronises itself through its blocking behaviour.

4.5 Shared-memory implementation with semaphores, and a message-queue variant

The benchmark program transfers *n* bytes with three methods and checks that the receiver computes the same checksum, so no data is lost or corrupted. The shared-memory version uses a ring of four 64 KB slots protected by two process-shared POSIX semaphores:

```
struct shm { sem_t empty, full; size_t len[SLOTS]; char data[SLOTS][CH]; };

/* producer (parent) */
int in = 0;
for (size_t o = 0; o < total; o += CH) {
    sem_wait(&s->empty);                 /* wait for a free slot */
    memcpy(s->data[in], src, CH);        /* fill it */
    s->len[in] = CH;
    sem_post(&s->full);                  /* announce a full slot */
    in = (in + 1) % SLOTS;
}
sem_wait(&s->empty); s->len[in] = 0; sem_post(&s->full);   /* length 0 = end
of data */

/* consumer (child) */
for (;;) {
    sem_wait(&s->full);                  /* wait for data */
    size_t l = s->len[out];
    if (l == 0) break;                   /* end of data */
    sum += sum64(s->data[out], l);       /* use the data directly in shared
memory */
    out = (out + 1) % SLOTS;
    sem_post(&s->empty);                 /* give the slot back */
}
```

The structure lives in a mmap(MAP_SHARED | MAP_ANONYMOUS) region created before fork(), so parent and child see the same bytes. Unlike a pipe, end of data has to be signalled by the program itself (here a zero length). The message-queue variant uses System V msgsnd and msgrcv with 4 KB messages and a special message type to mark the end.

4.6 Growth investigation: transfer time for 1 MB, 10 MB and 100 MB

Each cell is the best of five runs. All three methods produced identical checksums, so every byte arrived exactly once.

Data size	Pipe (64 KB writes)	Shared memory + semaphores (64 KB slots)	Message queue (4 KB messages)
1 MB	0.3 ms (3,074 MB/s)	0.3 ms (3,340 MB/s)	0.6 ms (1,628 MB/s)
10 MB	2.2 ms (4,548 MB/s)	**1.6 ms** (6,287 MB/s)	5.4 ms (1,868 MB/s)
100 MB	20.1 ms (4,976 MB/s)	**14.4 ms** (6,967 MB/s)	53.7 ms (1,863 MB/s)

Evaluation.

- **Linear growth.** Going from 10 MB to 100 MB multiplies the time by about 9 (pipe), 9 (shared memory) and 10 (message queue): the cost is proportional to the amount of data, which is O(n). At 1 MB the fixed costs (creating the child, setting up the semaphores) are a visible share, so the throughput is lower.

- **Shared memory is fastest**, by about 40 percent over the pipe at 10 MB and 100 MB (6.3 and 7.0 GB/s against 4.5 and 5.0 GB/s). This matches the copy count: it saves one of the two kernel copies and the system call per chunk. It is not many times faster, because the producer still copies each chunk into the slot, both sides still touch every byte, and a semaphore operation is needed for every 64 KB chunk.
- **The message queue is the slowest**, about 2.7 times slower than the pipe at 100 MB. The default size limit of a System V message (8 KB) forced 4 KB messages: 100 MB needs 25,600 `msgsnd` plus 25,600 `msgrcv` calls, against 1,600 `write` plus 1,600 `read` calls for the pipe. The per-message system-call overhead dominates. Larger chunks would narrow the gap.
- **Caveat.** The machine has one CPU, so sender and receiver never ran at the same instant; the figures include the time of the hand-over switches. On a multi-core machine the two processes can overlap and all three methods should be faster, with shared memory still leading.

4.7 Choice of method for the weather station

The weather station produces a few readings per second, a few bytes each. At that rate the transfer cost is irrelevant and *simplicity and safety* decide: a **pipe** needs no explicit synchronisation, preserves order, blocks the sensor reader instead of losing readings when the analyser is slow, and signals end of data automatically. It is the right method for the sensor-to-analyser link.

Shared memory with semaphores is justified only if the data rate becomes large (for example full radar or camera frames at high frequency), where the saved copy matters. A message queue is preferable when the messages have types or priorities (for example urgent storm alarms that must overtake ordinary readings).

4.8 Alternative concept and complexity analysis: shared memory with a semaphore instead of a pipe

Solution for the weather readings. The sensor reader creates a shared region holding a small ring buffer of integers and two semaphores: `empty` (initially the buffer size) and `full` (initially 0). The reader does `wait(empty)`, stores a reading, `post(full)`. The analyser does `wait(full)`, takes a reading, `post(empty)`. A reading of -1, or a separate flag, marks the end of data, since there is no automatic end-of-file. If several writers were possible, a mutex around the index updates would also be required (as in Q5).

Time to transfer n readings (4 bytes each).

Property	Pipe	Shared memory + semaphores
Data copies through the kernel	Two per reading (4n bytes in, 4n bytes out)	None
System calls	One `write` and one `read` per reading, unless the readings are batched	Semaphore operations are user-space atomic operations; a system call (futex) is

Property	Pipe	Shared memory + semaphores
		needed only when a process must actually sleep or be woken
Total cost	O(n) with a larger constant (copies plus system calls)	O(n) with a smaller constant
Measured, bulk data	4.5 to 5.0 GB/s	6.3 to 7.0 GB/s (about 40 percent faster)

For n readings the two methods are both linear in n; the difference lies in the constant. Batching amortises the cost for both: transferring 64 KB per operation instead of 4 bytes divides the number of system calls or semaphore operations by 16,384.

Synchronisation needed. The pipe needs none from the programmer. Shared memory needs two semaphores (one counting empty slots, one counting full slots) and, with several writers or readers, a mutex around the index variables. Mistakes in this protocol cause lost readings, duplicates or deadlock. The extra speed buys extra responsibility.

4.9 Evidence checklist for Q4

Evidence expected	Where it is provided
Working pipe implementation with expected output	Section 4.2
End-of-data explained and hang demonstrated	Section 4.3
Shared-memory (and message-queue) implementation	Section 4.5
Measured transfer times for 1, 10 and 100 MB	Section 4.6
Justified choice of method	Section 4.7
Alternative concept analysed	Section 4.8

Q5. Synchronisation: Critical Sections, Semaphores and Monitors

5.1 Problem summary

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with one global lock the system becomes very slow. The only starting point given is the critical-section principle: a solution must guarantee **mutual exclusion**, **progress** and **bounded waiting**. The task is to turn that principle into a design, evaluate the standard tools (Peterson's solution, semaphores, monitors), test a bounded-buffer program, and decide how coarse or fine the locks should be.

5.2 From the principle to a design

Step 1. Find the critical section. The shared data is the seat map of a coach (and a count of free seats). The code that must not be interleaved is the *check-then-act* sequence: look for a free seat, mark it sold, update the count. If two agents both see seat 17 as free before either marks it, both sell it. This sequence is the critical section; everything else (browsing, printing the ticket) is the remainder section.

Step 2. Give every agent the structure *entry section, critical section, exit section, remainder section*.

```
loop:
  ENTRY    : acquire permission
  CRITICAL : check seat, mark seat sold, update count
  EXIT     : release permission
  REMAINDER: print ticket, other work
```

Step 3. Translate each of the three requirements into a rule for the entry and exit sections.

Requirement	Meaning	What it means for the booking system	Design rule
Mutual exclusion	At most one agent in the critical section for the same data	Two agents never sell the same seat	Entry must be atomic: testing and taking the permission cannot be split
Progress	If nobody is in the critical section, the choice of who enters next depends only on agents that want to enter, and cannot be postponed forever	An agent that is only browsing must never block a booking agent	The permission may depend only on the state of the lock and the waiting agents, not on idle agents
Bounded waiting	After an agent asks to enter, only a bounded number of other agents can enter before it	A busy train cannot starve one unlucky agent	Waiting agents are served in a fair order (FIFO queue or turn variable)

Step 4. Choose the mechanism. Software-only solutions (Peterson) show that the rules are achievable but do not work reliably on modern processors (5.3). Hardware atomic instructions and the semaphores or mutexes built on them give mutual exclusion directly; a queue of blocked agents inside the semaphore gives bounded waiting; and a monitor wraps the whole pattern so it cannot be forgotten (5.6). The deployment question (how many locks) is then an engineering trade-off (5.9).

5.3 Peterson's solution for two processes

```
/* shared */  bool flag[2] = {false, false};  int turn;
/* process i, with j = 1 - i */
flag[i] = true;           /* I want to enter           */
turn = j;                 /* but you go first if you want */
while (flag[j] && turn == j) ;   /* busy-wait */
   /* critical section */
flag[i] = false;          /* exit */
```

Mutual exclusion. Suppose both processes are in the critical section. Process 0 passed its test, so flag[1] was false or turn was 0; process 1 passed, so flag[0] was false or turn was 1. Both flags are true by then, so turn would have to be 0 *and* 1 at the same time. turn holds a single value (the last assignment wins), so this is impossible.

Progress. If process j is in its remainder section, flag[j] is false, so process i's loop condition is false and it enters at once. If both want to enter, turn decides, and exactly one of them proceeds. No decision depends on a process that is not interested.

Bounded waiting. When process j leaves and tries again, it sets turn = i before waiting, which gives process i priority. Process i therefore waits for at most one entry of process j.

Why it may fail on modern processors. The proof assumes that the instructions run in program order and that a write becomes visible to the other processor immediately. Real hardware does not guarantee this:

- A CPU places stores in a **store buffer** and may execute a later load (the read of flag[j]) *before* the earlier store (flag[i] = true) is visible to other cores. Both processes can then read the other's flag as false and both enter. This reordering of a store followed by a load is allowed even on x86 and is more liberal on ARM and POWER.
- The **compiler** may also reorder or remove the assignments, or keep flag[j] in a register so the loop never sees the change.

The repair is to use atomic variables with sequentially consistent ordering (C11 _Atomic with memory_order_seq_cst) or explicit memory fences. In practice nobody uses Peterson's algorithm; it works only for two processes, burns CPU while waiting, and is replaced by atomic hardware instructions (test-and-set, compare-and-swap) that mutexes and semaphores are built on. *This section is an analysis; the failure was not reproduced on the one-CPU test machine, where the reordering windows are far harder to hit.*

5.4 Semaphores protecting the seat counter

A semaphore is an integer S that can be changed only by two atomic operations:

```
wait(S):   S = S - 1;  if S < 0: block this process in S's queue
signal(S): S = S + 1;  if S <= 0: wake one process from S's queue
```

	Counting semaphore	Binary semaphore (mutex)
Range	0 to any N	0 or 1
Use	Count a pool of identical resources, such as free seats or empty buffer slots	Mutual exclusion around a critical section
Initial value	Number of resources	1

The seat counter can be protected in two ways.

```
/* (a) binary semaphore protecting a counter and the seat map */
sem_t seat_mutex;           /* initialised to 1 */
sem_wait(&seat_mutex);
if (free_seats > 0) { free_seats--; mark_seat(); sold = 1; }
sem_post(&seat_mutex);

/* (b) counting semaphore whose value IS the number of free seats */
sem_t seats;                /* initialised to the number of seats */
if (sem_trywait(&seats) == 0) sold = 1;   /* seat reserved */
else                          sold = 0;   /* sold out, no waiting */
```

Because a blocked process sleeps in the semaphore's queue rather than looping, no CPU time is wasted waiting, and a FIFO queue gives bounded waiting. Mutual exclusion comes from the atomic decrement: the counter cannot go below zero, so seats cannot be oversold.

5.5 The classical problems solved with semaphores

Bounded buffer (producer and consumer). One mutex protects the buffer indices; one counting semaphore counts empty slots and one counts full slots.

```
semaphores: mutex = 1, empty = N, full = 0
Producer:  wait(empty); wait(mutex); put item; signal(mutex); signal(full)
Consumer:  wait(full);  wait(mutex); take item; signal(mutex); signal(empty)
```

The order matters: waiting for `mutex` *before* `empty` would let a producer hold the mutex while sleeping on a full buffer, so no consumer could enter to make room: deadlock.

Readers and writers (reader preference). Many agents may *read* the seat availability at the same time, but a *writer* (a booking) needs exclusive access.

```
semaphores: mutex = 1, wrt = 1;   int readcount = 0
Reader:  wait(mutex); readcount++; if (readcount == 1) wait(wrt); signal(mutex);
        ... read ...
        wait(mutex); readcount--; if (readcount == 0) signal(wrt); signal(mutex)
Writer:  wait(wrt); ... write ...; signal(wrt)
```

The first reader locks writers out and the last reader lets them in. Weakness: a continuous stream of readers starves writers; a writer-preference variant fixes that but may starve readers. For ticketing, where the availability screen is queried far more than bookings happen, this pattern fits the read-mostly traffic, and a fairness rule (for example a turnstile that holds back new readers when a writer waits) keeps bookings from starving.

Dining philosophers. Five philosophers (five processes) share five chopsticks (five binary semaphores). Each needs two to eat.

```
Naive:     wait(chop[i]); wait(chop[(i+1)%5]); eat; signal both
```

If all five pick up their left chopstick at the same moment, each waits forever for the right one: deadlock. Three standard repairs:

```
1. Room semaphore:  room = 4;  wait(room); take both chopsticks; eat; release both;
signal(room)
   (at most four philosophers compete, so at least one can always get two chopsticks)
2. Asymmetry:      odd philosophers take the left chopstick first, even ones the right first
3. Take both or neither: pick up both chopsticks inside a critical section (or a monitor)
```

The room solution is deadlock-free and simple; solution 3 additionally avoids holding one resource while waiting for another, which is the same idea as removing the *hold-and-wait* condition in Q6.

5.6 Monitors with condition variables

A **monitor** is a module that bundles shared data with the only operations that may touch it. The language or library guarantees that *at most one thread is active inside the monitor at a time*, so mutual exclusion is automatic and cannot be forgotten. **Condition variables** let a thread wait *inside* the monitor until a condition becomes true: wait(c) releases the monitor and sleeps; signal(c) wakes one waiter. In POSIX the monitor is a mutex plus condition variables:

```
typedef struct { int buf[N], in, out, count;
                 pthread_mutex_t m; pthread_cond_t not_full, not_empty; } BB;
void put(BB *b, int v) {
   pthread_mutex_lock(&b->m);
   while (b->count == N) pthread_cond_wait(&b->not_full, &b->m);   /* re-check in a loop */
   b->buf[b->in] = v; b->in = (b->in + 1) % N; b->count++;
   pthread_cond_signal(&b->not_empty);
   pthread_mutex_unlock(&b->m);
}
int take(BB *b) {
   pthread_mutex_lock(&b->m);
   while (b->count == 0) pthread_cond_wait(&b->not_empty, &b->m);
   int v = b->buf[b->out]; b->out = (b->out + 1) % N; b->count--;
   pthread_cond_signal(&b->not_full);
   pthread_mutex_unlock(&b->m);
   return v;
}
```

Why a monitor is easier to get right.

- The condition (count == N) is written *in the code that waits for it*. With semaphores the state is hidden in the counter values and the reader must work out what they mean.
- There is no order of wait operations to get wrong, so the mutex-before-empty deadlock of section 5.5 cannot occur: pthread_cond_wait releases the mutex atomically while sleeping.
- The shared data and its lock live together, so no code path can touch the data without the lock.
- The while loop re-checks the condition after waking, which makes the code correct even if the thread was woken spuriously or another thread consumed the item first.

The price is that the condition variable has no memory: a signal with nobody waiting is lost, which is why the state (count) must be kept in a variable and tested.

5.7 Completing the bounded-buffer program (TODO 1 and TODO 2)

```
void *producer(void *arg)
{
   (void)arg;
   for (int item = 1; item <= ITEMS; item++) {
      sem_wait(&empty);                  /* TODO 1 */
```

```
        pthread_mutex_lock(&lock);
        buffer[in] = item;
        in = (in + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&full);
    }
    return NULL;
}
void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);                /* TODO 2 */
        pthread_mutex_lock(&lock);
        int item = buffer[out];
        out = (out + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&empty);
        consumed_sum += item;
    }
    return NULL;
}
```

Compiled with `gcc -Wall -pthread`, the program printed (matching the expected output):

```
items consumed: 20, sum = 210 (expected 210)
```

Run 100 times: all 100 runs printed `items consumed: 20, sum = 210 (expected 210)` (100 identical lines under `sort | uniq -c`). The sum 210 is 1 + 2 + ... + 20.

5.8 Testing with 1 and with 4 producers and consumers

A generalised version of the program takes the number of producers and consumers on the command line. Every item has a unique identifier (producer number x 1000 + item number), and the consumers count how often each identifier is seen. 'Consumed exactly once' means every identifier was seen once, none zero times and none twice. The accumulation into the sum is done with an atomic add because several consumers now update it.

Configuration	Items	Sum (expected)	Never consumed	Consumed twice	Verdict
1 producer, 1 consumer, mutex	1,000	500,500 (500,500)	0	0	Correct
4 producers, 4 consumers, mutex	4,000	8,002,000 (8,002,000)	0	0	**Correct**
4 + 4, mutex, repeated 100 times	4,000 each	always the expected sum	0	0	100 of 100 correct

The 4 + 4 test shows that every item is consumed exactly once.

Effect of removing the mutex. The mutex lines were removed (the semaphores kept). To make the race visible on a single CPU, a `sched_yield()` was placed at the point where the

shared index is read and before it is updated; this widens the window in which another thread can interleave. It simulates the unlucky timing that multi-core hardware produces by itself.

Configuration, no mutex	Sum (expected 8,002,000 for 4 + 4)	Never consumed	Consumed twice	Verdict
1 + 1	500,500 (500,500)	0	0	Correct
4 + 4, run 1	8,013,835	1,605	1,603	**Wrong**
4 + 4, run 2	7,989,232	1,017	1,016	**Wrong**
4 + 4, 100 runs				100 of 100 wrong

Evaluation. With one producer and one consumer the result stays correct even without the mutex, because the producer is the only writer of `in` and the consumer the only writer of `out`, and the two semaphores already order the hand-over of each slot. With several producers or consumers the semaphores are *not enough*: two producers can read the same value of `in`, write into the same slot (one item overwrites the other, so it is never consumed) and advance `in` once; two consumers can read the same `out` and take the same item twice. The numbers of 'never consumed' and 'consumed twice' are almost equal for that reason: every lost item is replaced by a duplicate. The counting semaphores protect the *number* of items; only the mutex protects the *indices*. Correct code therefore needs both.

5.9 Deployment decision: one lock, one lock per train or one lock per coach

The booking model has 50 trains with 10 coaches each and 72 seats per coach. Agents pick a random train and coach and book the first free seat. Four setups were tested.

Experiment A: a very short critical section (the seat search and update take well under a microsecond), 8 and 16 agents x 400,000 requests, one CPU:

Locking	8 agents: requests per second	16 agents: requests per second	Double-sold seats
No lock	15.1 million	19.3 million	0 with 8 agents, **1 with 16 agents**
One global lock	15.6 million	16.3 million	0
One lock per train	13.6 million	16.8 million	0
One lock per coach	15.3 million	13.0 million	0

The unlocked version sold one seat twice (tickets sold 36,001, seats occupied 36,000), proving that locking is needed. The three locked versions are within noise of each other: with a critical section of a few tens of nanoseconds and a single CPU there is no contention to remove, and the finer locks only add memory and cache cost. *This experiment cannot discriminate between the granularities.*

Experiment B: a realistic critical section. In a real booking system the seat is not confirmed in nanoseconds: the agent must write to a database or confirm a payment. This was modelled

by holding the lock for 500 microseconds of waiting while the seat is confirmed. 64 agents, 50 bookings each (3,200 bookings), five repeated runs:

Locking	Time for 3,200 bookings (five runs)	Bookings per second	Speed relative to one global lock
One global lock	1.98 to 2.02 s	about 1,600	1
One lock per train	0.064 to 0.071 s	about 48,000	about 30 times
One lock per coach	0.036 to 0.040 s	about 84,000	about 53 times

A 16-agent run showed the same order (global 1,605 per second, per train 21,300, per coach 25,000). The global lock behaves exactly as theory predicts: it admits one booking per 500 microseconds (at most 2,000 per second), however many agents or CPUs there are. That is the 'system becomes very slow' symptom. Locking by train lets bookings for different trains proceed together; locking by coach also separates the coaches of the same train, so two agents collide only when they pick the *same coach*.

Decision: one lock per coach, with these rules:

1. *Lock ordering.* A group booking that needs seats in several coaches takes the coach locks in a fixed order (train number, then coach number), and releases them together. A consistent order prevents the circular wait that would otherwise deadlock two group bookings (see Q6).
2. *Whole-train operations* (cancelling a train, printing a full chart) take all coach locks of the train in the same order, or use a read-write lock at train level.
3. *Hold the lock for as short a time as possible.* The best improvement is not finer locks but a shorter critical section: reserve the seat under the lock (microseconds), release the lock, then confirm the payment, and give the seat back if the payment fails or times out. This removes the 500-microsecond wait from the critical section altogether.

Defence. Per coach gave the highest measured throughput (about 53 times the global lock and about 1.8 times the per-train lock) and has the lowest collision probability (500 locks instead of 50 or 1). Its cost is 500 mutexes (a few kilobytes of memory) and the need for ordered locking in multi-coach bookings. If group bookings across coaches were the dominant traffic, per-train locking would be the simpler and safer choice. The global lock is acceptable only when the critical section is purely in memory and very short (Experiment A shows tens of millions of bookings per second), which is not the case once a database is involved.

5.10 Alternative concept and complexity analysis: atomic operations or a spin lock instead of a semaphore and mutex

Solution with atomics. For a pure counter of free seats, `atomic_fetch_add` (or a compare-and-swap loop) needs no lock:

```
atomic_int seats_left = 72;
int take_seat(void) {
   int old = atomic_load(&seats_left);
   while (old > 0 && !atomic_compare_exchange_weak(&seats_left, &old, old - 1))
      ;                    /* old is reloaded on failure; retry */
```

```
    return old > 0;          /* 1 = seat reserved, 0 = sold out */
}
```

Solution with a spin lock: while (atomic_flag_test_and_set(&sp)) ; before the update and atomic_flag_clear(&sp) after it.

Measured cost of one update (1,000,000 updates per thread; one CPU):

Method	1 thread	4 threads	16 threads	CPU time for 16 threads (wall time)
atomic_fetch_add	6.9 ns	6.8 ns	6.8 ns	0.108 s (0.109 s)
Spin lock	9.1 ns	23.1 ns	**64.9 ns**	**1.029 s** (1.038 s)
Mutex	23.5 ns	25.0 ns	23.8 ns	0.374 s (0.382 s)

All variants produced the exact final count (for example 16,000,000 with 16 threads).

Evaluation.

- **Atomic operation.** Fastest and flat as threads increase: a single hardware instruction, no waiting, no kernel call. It protects exactly *one* variable. The booking has several variables (seat map and counter) that must change together, so an atomic counter alone cannot keep the seat map consistent.
- **Spin lock.** Cheap when uncontended (9 ns), but a waiting thread burns CPU. With 16 threads on one CPU the spin lock consumed about **1.0 s of CPU for 16 million updates, almost 10 times as much as the atomic** (0.108 s) and nearly three times the mutex (0.374 s), because a thread that is pre-empted *inside* the critical section leaves the others spinning uselessly until it runs again. On several cores a spin lock can still win for critical sections of a few instructions, but only if the lock holder cannot be pre-empted.
- **Mutex or semaphore.** About 24 ns per update regardless of threads, because a blocked thread sleeps and does not use CPU. For a **long critical section** (anything involving I/O, a database call or more than a few microseconds) a blocking mutex or semaphore is the right method; spinning would waste a whole core for the entire duration.

Which method suits which part of the booking system. Atomic operations for statistics counters and the free-seat count if it stands alone; a mutex per coach for the seat map update; a semaphore or condition variable where threads must wait for a condition (for example a waiting list); a spin lock only in short, non-pre-emptible kernel-style code.

5.11 Evidence checklist for Q5

Evidence expected	Where it is provided
Student-derived synchronisation design	Section 5.2, with 5.3 to 5.6 for the tools
Working semaphore code	Sections 5.5 and 5.7
Test evidence (1 and 4 threads, 100 runs, mutex removed)	Sections 5.7 and 5.8
Measured, defended locking decision	Section 5.9
Alternative with atomics and spin locks	Section 5.10

Q6. Deadlock Handling and the Banker's Algorithm

6.1 Problem summary

A research centre runs jobs that need several shared resources at once: GPUs, licences for simulation software and high-speed storage. Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes. The question has six parts: A characterise deadlock (2 marks), B compare the strategies (3), C apply the Banker's algorithm (4), D predict the running time (1), E measure it (1) and F defend the final design (2). Part C is presented before B, D and E because the comparison and the measurements rely on it.

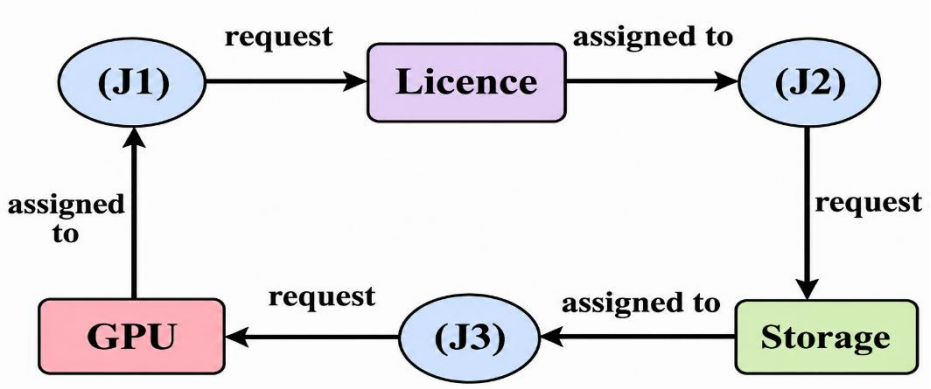
Part A. Characterise deadlock

The four necessary conditions. A deadlock can occur only if all four hold at the same time:

Condition	Meaning	Where it appears in the cluster
1. Mutual exclusion	At least one resource can be used by only one job at a time	A GPU, a single-seat simulation licence and an exclusive storage volume cannot be shared
2. Hold and wait	A job holds some resources while waiting for others	A job keeps its GPU while waiting for a licence
3. No pre-emption	A resource cannot be taken away from a job; it is released only voluntarily	A running GPU job cannot be stopped and its GPU removed without losing its work
4. Circular wait	A closed chain of jobs exists in which each waits for a resource held by the next	J1 waits for J2, J2 for J3, J3 for J1

Resource-allocation graph for the cluster. Jobs are circles, resources are boxes. An arrow from a job to a resource is a *request*; an arrow from a resource to a job is an *assignment*. Take three jobs and three single-instance resources:

- J1 holds the GPU and requests the Licence.
- J2 holds the Licence and requests the Storage volume.
- J3 holds the Storage volume and requests the GPU.



The graph contains the cycle J1, Licence, J2, Storage, J3, GPU, J1. Because every resource has a *single* instance, a cycle in the graph means a deadlock is certain: nothing in the cycle can ever proceed.

Condition	Evidence in the graph
Mutual exclusion	Each box has one instance, held by exactly one job at a time
Hold and wait	Every job has an incoming assignment arrow (it holds something) and an outgoing request arrow (it wants more)
No pre-emption	No arrow can be removed except by the holder releasing the resource, and the holder cannot release because it is blocked
Circular wait	The closed loop J1, Licence, J2, Storage, J3, GPU

If a resource had several instances (for example four GPUs) a cycle would show only a *possible* deadlock, because a job outside the cycle might release an instance; in that case the safety algorithm of Part C is needed.

Part C. Apply the Banker's algorithm (4 marks)

C.1 The data

Five processes (P0 to P4) and three resource types A, B, C. The numbers are those of the program supplied with the question.

Process	Allocation (A B C)	Maximum (A B C)	Need = Max - Alloc
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Available = (3, 3, 2).

C.2 The two algorithms

Safety algorithm. (1) Set Work = Available and mark all processes unfinished. (2) Find an unfinished process whose Need is less than or equal to Work in every resource. (3) If one exists, assume it runs to completion and returns its allocation: Work = Work + Allocation, mark it finished, and repeat from step 2. (4) If none exists, the state is safe if and only if every process is finished. The order in which the processes finished is a **safe sequence**.

Resource-request algorithm for process Pi requesting vector Request: (1) if Request exceeds Need, error (the process exceeded its declared maximum); (2) if Request exceeds Available, Pi must wait; (3) otherwise *pretend* to allocate (Available minus Request, Allocation plus Request, Need minus Request) and run the safety algorithm; if the new state is safe, keep it; if not, restore the old state and make Pi wait.

C.3 Implementation (completing TODO 1)

```
int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;
            int ok = 1;                                  /* TODO 1 */
            for (int r = 0; r < R; r++)
                if (maxm[p][r] - alloc[p][r] > work[r]) { ok = 0; break; }
            if (ok) {
                for (int r = 0; r < R; r++) work[r] += alloc[p][r];
                finish[p] = 1;
                seq[count++] = p;
                found = 1;
            }
        }
        if (!found) return 0;
    }
    return 1;
}
```

The rest of the program (the request() function and main) is the code given in the question and was not changed.

Output (identical to the expected output):

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

C.4 Hand verification of the safe sequence

Work = (3, 3, 2):

Step	Process chosen	Need versus Work	New Work
1	P1	(1,2,2) is at most (3,3,2)	(3,3,2) + (2,0,0) = (5,3,2)
2	P3 (P2 skipped: needs 6 of A,	(0,1,1) is at most (5,3,2)	(5,3,2) + (2,1,1) = (7,4,3)

Step	Process chosen	Need versus Work	New Work
	only 5 available)		
3	P4	(4,3,1) is at most (7,4,3)	(7,4,3) + (0,0,2) = (7,4,5)
4 (second scan)	P0	(7,4,3) is at most (7,4,5)	(7,4,5) + (0,1,0) = (7,5,5)
5	P2	(6,0,0) is at most (7,5,5)	(7,5,5) + (3,0,2) = (10,5,7)

Safe sequence: **P1, P3, P4, P0, P2**, as in the program. The final Work (10, 5, 7) equals the total of all resources (initial Available (3,3,2) plus all allocations (7,2,5)), which is a useful check.

C.5 Decision for each request

Request 1: P1 asks for (1, 0, 2). Granted. Check 1: (1,0,2) is at most Need of P1, (1,2,2). Check 2: (1,0,2) is at most Available (3,3,2). Pretend to allocate: Available = (2,3,0), Allocation of P1 = (3,0,2), Need of P1 = (0,2,0). Safety with Work = (2,3,0): P1 (needs (0,2,0)) can finish, Work = (5,3,2); P3, P4, P0 and P2 then follow exactly as in the table above. The new state is safe with sequence P1, P3, P4, P0, P2, so the request is **granted**.

Request 2: P4 asks for (3, 3, 0). Must wait. Check 1: (3,3,0) is at most Need of P4, (4,3,1), so it is legal. Check 2: Available is now (2,3,0). The request wants 3 units of A, only 2 are free, so P4 must **wait** (no pretend allocation is made). This is a shortage, not a safety problem.

Request 3: P0 asks for (0, 2, 0). Denied (unsafe). Check 1: (0,2,0) is at most Need of P0, (7,4,3). Check 2: it is at most Available (2,3,0). Pretend to allocate: Available = (2,1,0), Allocation of P0 = (0,3,0), Need of P0 = (7,2,3). Now the needs are:

Process	Need	Can it finish with Work = (2, 1, 0)?
P0	(7,2,3)	No: needs 7 of A (only 2)
P1	(0,2,0)	No: needs 2 of B (only 1)
P2	(6,0,0)	No: needs 6 of A (only 2)
P3	(0,1,1)	No: needs 1 of C (none available)
P4	(4,3,1)	No: needs 4 of A (only 2)

No process can finish, so no process can ever return resources, and the state is **unsafe**. The request is **denied** and the state is restored (Available back to (2,3,0), P0's allocation back to (0,1,0)). The instructive point: the resources *are* physically free, but granting them removes one unit of B that P1 needs and leaves no unit of C for P3, so every process would be waiting for something that nobody can supply. An unsafe state is not yet a deadlock, but the system has lost its guarantee of avoiding one.

Part B. Compare the strategies (3 marks)

Strategy	How it works	Cost	Strengths	Weaknesses	Fit for the cluster
Prevention (remove one of the four conditions)	Mutual exclusion: make resources shareable. Hold and wait: a job requests everything at once, or releases all before asking again. No pre-emption: allow resources to be taken back. Circular wait: give resources a global order and let jobs request only in increasing order	Very low at run time; costs utilisation	Simple, deterministic, no run-time checks	Mutual exclusion cannot be removed for GPUs or licences. All-at-once allocation wastes resources while a job holds unused ones. GPU pre-emption loses work unless checkpointed. Ordering must be obeyed by every job	Resource ordering and all-at-once allocation are practical; the other two are not
Avoidance (Banker's algorithm)	Each job declares its maximum demand; every request is granted only if the resulting state is safe	O(m x n squared) per request in the worst case (Part E)	Never enters an unsafe state; allows more concurrency than prevention; uses resources well	Needs maximum demands in advance; fixed set of jobs and resource types; conservative (may delay safe requests); the cost of every check	Good if jobs declare needs when submitted, as batch jobs do
Detection and recovery	Let deadlocks happen; run a detection algorithm periodically	No cost per request; detection O(m x n	Maximum concurrency; no prior knowledge	Work is lost when a job is aborted; the	Reasonable as a safety net, expensive

Strategy	How it works	Cost	Strengths	Weaknesses	Fit for the cluster
	(resource-allocation graph or matrix algorithm); recover by aborting jobs or pre-empting resources	squared) every few minutes	needed; handles surprises	cluster is stuck until the next detection; recovery policy is complex	for long jobs without checkpoints
Ignore (the ostrich approach)	Assume deadlock is rare; the administrator reboots or kills jobs when it happens	None	Zero overhead; used by most general-purpose systems for rare cases	The cluster stops until a human intervenes; the cost of a stop grows with the value of the jobs	This is what the cluster does now, and it is the problem

Judgement for this environment. The ostrich approach has already failed: deadlocks are frequent enough to stop the cluster. Prevention by removing mutual exclusion or pre-emption is not feasible for GPUs and licences; prevention by all-at-once allocation or resource ordering is feasible but can waste expensive hardware. Detection with recovery keeps utilisation high but wastes the hours of computation of an aborted job. Avoidance fits well, because batch jobs *can* declare their maximum needs when they are submitted, the number of jobs is moderate, and the cost of a safety check is tiny compared with the minutes or hours that a job runs (Part E).

Part D. Predict before implementing (1 mark)

Analysis of the safety algorithm. The inner test of one process compares at most m numbers: O(m). The for loop tests up to n processes: O(m x n). The outer loop repeats until all processes have finished, and each pass finishes at least one, so there are up to n passes. The worst case is therefore **O(m x n squared)**. The best case (every process can finish in the first pass) is one pass: **O(m x n)**.

Prediction (testable).

1. In the worst case, multiplying n by 10 multiplies the running time by about 100; multiplying n by 4 multiplies it by about 16.
2. In the best case, multiplying n by 10 multiplies the time by about 10 (linear).
3. The time grows at most linearly with m (doubling m at most doubles the time), and in the worst case it can grow less, because the comparison stops at the first resource that does not fit.
4. Consequently, if every request triggers a worst-case check, the number of requests per second the system can decide falls with the square of the number of jobs.

Part E. Measure (1 mark)

Method. The safety algorithm was rewritten for arbitrary n and m (dynamic arrays) and timed with a monotonic clock, repeating each measurement enough times to get a stable average (200,000 repetitions for n = 10 down to 3 for n = 4,000). Two states were constructed. *Worst case:* process n-1 can finish first, then n-2, and so on, so the while loop needs n passes. *Best case:* every process can finish in the first pass. Every constructed state was safe (checked in the program).

n (jobs)	m (resources)	Worst case	Best case	Worst-case checks per second
10	3	0.1 us	0.0 us	about 8,400,000
100	3	8.9 us	0.4 us	about 112,000
1,000	3	715 us	3.3 us	about 1,400
4,000	3	10,791 us	13.3 us	about 93
10	6	0.2 us	0.1 us	about 6,500,000
100	6	8.6 us	0.9 us	about 116,000
1,000	6	690 us	7.7 us	about 1,450
4,000	6	10,783 us	30.9 us	about 93

Comparison with the prediction.

Prediction	Measurement	Verdict
Worst case grows 100 times for 10 times more jobs	n 100 to 1,000: 8.9 to 715 us, a factor of **80**	Confirmed (the smaller n has some fixed overhead)
Worst case grows 16 times for 4 times more jobs	n 1,000 to 4,000: 715 to 10,791 us, a factor of **15.1**	Confirmed
Best case grows linearly	n 100 to 1,000: 0.4 to 3.3 us (8 times); n 1,000 to 4,000: 3.3 to 13.3 us (4 times)	Confirmed
At most linear growth in m	Best case, n = 1,000: m = 3 to 6 gives 3.3 to 7.7 us (2.3 times); worst case unchanged (715 against 690 us)	Confirmed. The worst-case construction fails at the first resource, so the inner loop exits at once; the best case must check all m

The measured growth follows the predicted O(m x n squared) for the worst case and O(m x n) for the best case. For a realistic cluster of 100 to 1,000 jobs a safety check costs between a fraction of a microsecond and 0.7 ms.

Part F. Defend the final design

Recommendation for the research centre: avoidance by the Banker's algorithm as the admission rule for resource requests, with resource ordering inside the licence pool and a periodic detection run as a safety net.

1. **Primary strategy: the Banker's algorithm.** Each job must state its maximum demand (GPUs, licences, storage volumes) in its submission script. A request is granted only if the resulting state is safe; otherwise the job waits. The cluster then *cannot* enter a deadlock caused by the modelled resources, which removes the stops that require an administrator.
2. **Cost of the Banker's algorithm.** The measured worst case is 0.7 ms for 1,000 jobs and 10.8 ms for 4,000 jobs; the typical case is several microseconds. A job that runs for minutes or hours is delayed by a negligible fraction (about one millionth of its run time). The cost grows with the square of the number of jobs, so for a cluster with tens of thousands of simultaneous jobs the check would need to be restricted to the jobs that compete for the same resource pool, or run on a faster path (for example, a quick test that the free resources cover the largest remaining need).
3. **The need to know maximum demands in advance.** This is the main limitation, and it is acceptable here because batch jobs are submitted with resource requests; a scheduler already needs them to place the job. Two rules make it safe: the declared maximum is *enforced* (a job that asks for more than it declared is rejected or terminated), and every job must eventually release its resources (jobs have a time limit). Without those rules the safety proof does not hold.
4. **Other limits to accept.** Avoidance is conservative: it sometimes delays a request that would have been harmless, which lowers utilisation slightly compared with letting everything run. It also assumes a fixed number of resource types; adding a new GPU pool means updating the model.
5. **Safety net.** Detection by the matrix algorithm every few minutes (cost under a millisecond for 1,000 jobs, so negligible) catches deadlocks caused by something outside the model, such as a licence server crash. The recovery rule is to abort the youngest job holding the fewest resources, with checkpoints reducing the lost work.
6. **Why not the alternatives alone.** Detection alone would still stop the cluster until the next run and would waste hours of GPU time each time it aborted a job. Prevention by all-at-once allocation is simple and deadlock-free but idles expensive GPUs while jobs wait for licences. Ignoring deadlock is what created the problem.

Alternative concept and complexity analysis: detection and recovery instead of the Banker's algorithm

Solution. Every request is granted immediately if the resource is free. A detection algorithm (the same structure as the safety algorithm, but using the *current requests* of the waiting jobs instead of their maximum needs) runs every few minutes. Jobs that cannot finish even if everything else did are *deadlocked*; one or more are aborted (or rolled back to a checkpoint) until the cycle is broken.

Aspect	Banker's algorithm	Detection and recovery
Cost per request	O(m x n squared) worst case, measured 0.7 ms for n = 1,000, about 3 us typically	O(m) (allocate if free): effectively **no check**
Periodic cost	None	One detection run, O(m x n squared), about 0.7 ms for n = 1,000 every few minutes: under one part per hundred thousand of the CPU
Information needed	Maximum demand of every job in advance	Current allocation and current requests only
Utilisation	Slightly lower (conservative)	Highest
Time spent in deadlock	None	Up to the detection interval (minutes), during which the deadlocked jobs and the resources they hold are idle
Work lost	None	A deadlocked job's computation since its last checkpoint; hours of GPU time without checkpoints

If deadlocks were truly rare, detection would be cheaper. For this cluster, where they happen often enough to halt operations and each aborted job can represent hours of work, the measured cost of the Banker's algorithm is small enough that avoiding the deadlock is better than repairing it.

6.2 Evidence checklist for Q6

Part	Evidence expected	Where it is provided
A	Correct resource-allocation graph identifying all four conditions	Part A
B	Reasoned comparison of the four strategies	Part B
C	Working implementation with correct safe sequences and decisions	Part C (output, hand check, three decisions)
D	Clear, testable prediction	Part D
E	Measured times compared with the prediction	Part E

Part	Evidence expected	Where it is provided
F	Recommendation justified, including cost and maximum-demand requirement	Part F

---

7. Overall Conclusions

7.1 Summary of the findings

Question	Main finding	Decision
Q1 Processes	At 2,000 sessions a process costs about 152 KB of kernel bookkeeping and 157 to 197 us to create, against 16 to 33 KB and 19 to 20 us for a thread; sessions mostly wait, and the hangs come from run-queue length and memory pressure	A fixed pool of worker processes with an event loop or small thread pool in each
Q2 Scheduling	SRTF is best on averages (WT 6.50) but needs burst times; Round Robin degenerates to FCFS at a large quantum; on the video workload thumbnails wait 3,210 s under FCFS and 0.45 s under a multilevel feedback queue	Multilevel feedback queue with ageing and FCFS in the lowest level
Q3 Threads	Speed-up saturates at about 2.1 on one CPU because the compute part cannot be shared; an unprotected counter lost 64 percent of its updates	Kernel-thread pool sized cores x (1 + wait/compute), atomics or mutexes for shared totals
Q4 IPC	Shared memory is about 40 percent faster than a pipe for bulk data, a message queue with small messages 2.7 times slower; a pipe hangs if the writer does not close its end	A pipe for the sensor link; shared memory with semaphores only for high-rate data
Q5 Synchronisation	Semaphores alone are not enough with several producers or consumers (100 of 100 wrong runs without the mutex); with a 500-microsecond critical section a per-coach lock is about 53 times faster than one global lock	One lock per coach with ordered locking; keep critical sections short
Q6 Deadlock	P0's request for (0,2,0) leads to an unsafe state although resources are free; the safety check grows as n squared (0.7 ms at n = 1,000)	Banker's algorithm with declared and enforced maxima, plus periodic detection as a safety.

7.2 Common themes

- **Isolation against sharing is the recurring trade-off.** Processes isolate faults but cost memory and time; threads share cheaply but share failures and data races; shared memory is fast but needs locks.
- **A bounded pool beats unbounded creation.** Pools of processes (Q1), threads (Q3) and workers limit resource use and keep latency predictable under load.
- **Correctness first, speed second.** Every fast variant in this report that lacked synchronisation (unprotected counter, unlocked ticket sales, buffer without a mutex) gave wrong answers.
- **Measure the real critical section.** The lock-granularity question (Q5) could not be settled by a nanosecond critical section; it needed the realistic hold time.

7.3 Limitations of the evidence

- The test machine had **one virtual CPU**. Results that depend on true parallelism were explained rather than demonstrated, and multi-core behaviour is stated only as an expectation.
- Timings come from short runs on a shared virtual machine and vary by several percent; the ratios and orders of magnitude are reliable, the last digits are not.
- The priorities used for the priority-scheduling table in Q2 were assumed, because the question did not provide them.
- The failure of Peterson's algorithm on modern hardware (Q5) is explained, not reproduced.
- The scheduling-latency figure in Q1 is an analytical estimate.

7.4 How to reproduce the experiments

All programs were compiled with gcc on Linux. Typical commands:

```
gcc -O2 -pthread -o q1_fork q1_fork.c && ./q1_fork 4
gcc -O2 -o q2_rr q2_rr.c && ./q2_rr 3        # quantum 1, 3, 5, 20
python3 q2_sched.py                          # five algorithms and the video workload
gcc -O1 -pthread -o q3_threads q3_threads.c && ./q3_threads
gcc -Wall -o q4_pipe q4_pipe.c && ./q4_pipe
gcc -O2 -pthread -o q4_bench q4_bench.c && ./q4_bench
gcc -Wall -pthread -o q5_bb q5_bb.c && ./q5_bb
gcc -O1 -pthread -o q5_multi q5_multi.c && ./q5_multi 4 1   # 4 producers and consumers, mutex on
gcc -O2 -pthread -o q5_lock q5_lock.c && ./q5_lock 8
gcc -Wall -o q6_banker q6_banker.c && ./q6_banker
gcc -O2 -o q6_bench q6_bench.c && ./q6_bench
```

7.5 References

- Silberschatz, A., Galvin, P. B. and Gagne, G., *Operating System Concepts*, Wiley: chapters on processes, threads, CPU scheduling, synchronisation and deadlocks.
- Tanenbaum, A. S. and Bos, H., *Modern Operating Systems*, Pearson.
- Arpaci-Dusseau, R. H. and Arpaci-Dusseau, A. C., *Operating Systems: Three Easy Pieces*, free online (multilevel feedback queue, locks and concurrency chapters).

- The Linux manual pages for fork(2), pipe(2), pthreads(7), sem_overview(7), shm_overview(7) and proc(5).

Appendix A. Supporting Source Code for Q1 to Q3

The listings below are the measurement programs behind the tables in the report. The main answer programs (fork demo, Round Robin, pipe, bounded buffer, Banker's algorithm) are already shown in full in their sections.

A.1 Q1: creation cost and memory of 2,000 processes or threads (q1_scale.c)

Usage: ./q1_scale proc 2000 or ./q1_scale thread 2000. The parent first touches a 20 MB heap so that fork() has realistic page tables to copy.

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <time.h>
#include <sys/wait.h>
static double now(void){struct timespec
t;clock_gettime(CLOCK_MONOTONIC,&t);return t.tv_sec+t.tv_nsec*1e-9;}
static long memavail_kb(void){FILE*f=fopen("/proc/meminfo","r");char
l[128];long v=0;

while(fgets(l,128,f))if(!strncmp(l,"MemAvailable:",13)){sscanf(l+13,"%ld",&v
);break;}
    fclose(f);return v;}
static void *th(void*a){(void)a;sleep(3);return NULL;}
int main(int argc,char**argv){
    int n=atoi(argv[2]); int proc=!strcmp(argv[1],"proc");
    pid_t *k=malloc(n*sizeof(pid_t)); pthread_t
*t=malloc(n*sizeof(pthread_t));
    char *heap=malloc(20<<20); memset(heap,1,20<<20);       /* 20 MB server
heap */
    long m1=memavail_kb(); double t0=now();
    if(proc){
        for(int i=0;i<n;i++){pid_t p=fork();
if(p<0){perror("fork");n=i;break;}
                             if(p==0){sleep(3);_exit(0);} k[i]=p;}
    } else {
        pthread_attr_t
a;pthread_attr_init(&a);pthread_attr_setstacksize(&a,64*1024);
        for(int
i=0;i<n;i++){if(pthread_create(&t[i],&a,th,NULL)){perror("pthread_create");n
=i;break;}}
    }
    double t1=now(); usleep(500000); long m2=memavail_kb();
    printf("%s n=%d create_total=%.1f ms per_create=%.1f us extra_mem=%ld MB
(%.0f KB each)\n",
        argv[1],n,(t1-t0)*1e3,(t1-t0)*1e6/n,(m1-m2)/1024,(double)(m1-m2)/n);
    if(proc)for(int i=0;i<n;i++)waitpid(k[i],NULL,0); else for(int
i=0;i<n;i++)pthread_join(t[i],NULL);
    return 0;}
```

Raw output (two runs of each):

```
thread n=2000 create_total=38.1 ms per_create=19.0 us extra_mem=64 MB (33 KB each)
proc   n=2000 create_total=394.2 ms per_create=197.1 us extra_mem=296 MB (152 KB each)
proc   n=2000 create_total=313.1 ms per_create=156.5 us extra_mem=302 MB (155 KB each)
thread n=2000 create_total=40.5 ms per_create=20.2 us extra_mem=32 MB (16 KB each)
```

A.2 Q1: context-switch time with a pipe ping-pong (q1_ctx.c)

Usage: ./q1_ctx p (two processes) or ./q1_ctx t (two threads). Both sides are pinned to CPU 0, so every message forces a switch. Time per switch is the total time divided by twice the number of round trips.

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sched.h>
#include <pthread.h>
#include <time.h>
#include <sys/wait.h>
#define IT 200000
static double now(void){struct timespec
t;clock_gettime(CLOCK_MONOTONIC,&t);return t.tv_sec+t.tv_nsec*1e-9;}
static int a[2],b[2];
static void pin(void){cpu_set_t
s;CPU_ZERO(&s);CPU_SET(0,&s);sched_setaffinity(0,sizeof s,&s);}
static void *echo(void*x){(void)x;char c;pin();
    for(int i=0;i<IT;i++){read(a[0],&c,1);write(b[1],&c,1);}return NULL;}
int main(int argc,char**argv){
    pipe(a);pipe(b);pin();char c='x';
    if(argv[1][0]=='p'){
        pid_t p=fork(); if(p==0){echo(NULL);_exit(0);}
        double t0=now();for(int
i=0;i<IT;i++){write(a[1],&c,1);read(b[0],&c,1);}
        double t=now()-t0;waitpid(p,NULL,0);
        printf("processes: %.2f us per switch\n",t/(2.0*IT)*1e6);
    } else {
        pthread_t th;pthread_create(&th,NULL,echo,NULL);
        double t0=now();for(int
i=0;i<IT;i++){write(a[1],&c,1);read(b[0],&c,1);}
        double t=now()-t0;pthread_join(th,NULL);
        printf("threads:   %.2f us per switch\n",t/(2.0*IT)*1e6);
    }
    return 0;}
```

Raw output: processes 1.26 us and 1.16 us; threads 1.09 us and 1.07 us.

A.3 Q2: scheduling simulator (q2_sched.py, core functions)

The simulator implements FCFS, non-preemptive SJF, SRTF, non-preemptive priority, Round Robin and a three-level multilevel feedback queue. Each returns the Gantt segments, the completion time and the first start time of each job, from which TAT, WT and RT are computed.

```
from collections import deque
def metrics(jobs, fin, first):
    out = []
    for j in jobs:
        tat = fin[j['n']] - j['at']; wt = tat - j['bt']; rt = first[j['n']]
```

```
- j['at']
        out.append((j['n'], fin[j['n']], tat, wt, rt))
    return out

def np_pick(jobs, key):                  # FCFS, SJF and priority differ only
in the key
    t = 0; g = []; fin = {}; first = {}; rem = list(jobs)
    while rem:
        av = [j for j in rem if j['at'] <= t]
        if not av: t = min(j['at'] for j in rem); continue
        j = min(av, key=lambda j: (key(j), j['at'], j['n']))
        first[j['n']] = t; g.append((j['n'], t, t + j['bt'])); t += j['bt']
        fin[j['n']] = t; rem.remove(j)
    return g, fin, first
fcfs = lambda jobs: np_pick(jobs, lambda j: j['at'])
sjf  = lambda jobs: np_pick(jobs, lambda j: j['bt'])
prio = lambda jobs: np_pick(jobs, lambda j: j['pr'])

def srtf(jobs):                           # one time unit at a time
    rem = {j['n']: j['bt'] for j in jobs}; t = 0; g = []; fin = {}; first =
{}
    while rem:
        av = [j for j in jobs if j['n'] in rem and j['at'] <= t]
        if not av: t = min(j['at'] for j in jobs if j['n'] in rem); continue
        j = min(av, key=lambda j: (rem[j['n']], j['at'], j['n']))
        first.setdefault(j['n'], t); g.append((j['n'], t, t + 1));
rem[j['n']] -= 1; t += 1
        if rem[j['n']] == 0: del rem[j['n']]; fin[j['n']] = t
    return merge(g), fin, first

def rr(jobs, q):
    jobs = sorted(jobs, key=lambda j: j['at']); left = {j['n']: j['bt'] for
j in jobs}
    t = 0; g = []; fin = {}; first = {}; dq = deque(); i = 0
    while len(fin) < len(jobs):
        while i < len(jobs) and jobs[i]['at'] <= t: dq.append(jobs[i]['n']);
i += 1
        if not dq: t = jobs[i]['at']; continue
        p = dq.popleft(); first.setdefault(p, t); run = min(q, left[p])
        g.append((p, t, t + run)); t += run; left[p] -= run
        while i < len(jobs) and jobs[i]['at'] <= t: dq.append(jobs[i]['n']);
i += 1
        if left[p] > 0: dq.append(p)          # arrivals first, then the
pre-empted job
        else: fin[p] = t
    return g, fin, first
```

The multilevel feedback queue keeps three FIFO queues with quanta 4, 16 and 64 time units; a new job enters the top queue, a job that uses its whole quantum moves one level down, and a job in a higher queue pre-empts the running job of a lower queue. The starvation demonstration runs the same priority selection with and without a priority bonus of one level per 10 (or 5) time units of waiting.

Job sets used. Part 1 and the quantum study: P1 (arrival 0, burst 8, priority 3), P2 (1, 4, 1), P3 (2, 9, 4), P4 (3, 5, 2). Video workload: three films of 1,200 units arriving at 0, 10 and 20, and 20 thumbnails of 2 units arriving at 1, 6, 11, and so on (every 5 units). Starvation demonstration: 40 jobs of burst 5 and priority 1 arriving every 4 units from time 0, and one job L of burst 10 and priority 9 arriving at time 2.

A.4 Q3: camera-processing and counter-race program (q3_threads.c, main routine)

The helper functions process_camera, pool_worker, one and run_pool are listed in section 3.4. The main routine times the five variants and the three counter experiments.

```
int main(void){
    double t0=now();
    for(int c=0;c<CAMERAS;c++) result[c]=process_camera(c);
    double seq=now()-t0;
    printf("sequential (1 thread)       : %.3f s  speed-up 1.00\n",seq);
    t0=now(); pthread_t t[CAMERAS];
    for(int c=0;c<CAMERAS;c++)
pthread_create(&t[c],NULL,one,(void*)(intptr_t)c);
    for(int c=0;c<CAMERAS;c++) pthread_join(t[c],NULL);
    double tp=now()-t0;
    printf("thread per camera (64)      : %.3f s  speed-up
%.2f\n",tp,seq/tp);
    int sizes[]={4,8,16};
    for(int k=0;k<3;k++){double tt=run_pool(sizes[k]);
        printf("pool of %2d threads           : %.3f s  speed-up
%.2f\n",sizes[k],tt,seq/tt);}
    /* 4 worker processes, results returned through a pipe */
    int fd[2]; pipe(fd); t0=now();
    for(int k=0;k<4;k++) if(fork()==0){
        close(fd[0]);
        for(int c=k;c<CAMERAS;c+=4){uint64_t
r=process_camera(c);write(fd[1],&r,sizeof r);}
        _exit(0);}
    close(fd[1]); uint64_t r; int got=0;
    while(read(fd[0],&r,sizeof r)==sizeof r) got++;
    while(wait(NULL)>0);
    double tpr=now()-t0;
    printf("4 worker processes + pipe   : %.3f s  speed-up %.2f (%d
results)\n",tpr,seq/tpr,got);
    /* shared counter: unsafe / mutex / atomic, 4 threads x 2,000,000
increments */
    return 0;}
```

The three counter functions are the ones described in section 3.6: unsafe_cnt++ on a volatile long, mutex_cnt++ between pthread_mutex_lock and pthread_mutex_unlock, and atomic_fetch_add(&atomic_cnt, 1).

Appendix B. Supporting Source Code and Raw Output for Q4 to Q6

B.1 Q4: transfer benchmark (q4_bench.c, pipe and message-queue functions)

The shared-memory function is listed in section 4.5. Each method returns the elapsed time and a checksum of everything the receiver saw; the main routine takes the best of five runs and checks that the three checksums are equal.

```
#define CH 65536
static uint64_t sum64(const void*p,size_t n){const uint64_t*w=p;uint64_t
s=0;
    for(size_t i=0;i<n/8;i++)s+=w[i];return s;}
static double by_pipe(size_t total,uint64_t*chk){
    int fd[2];pipe(fd);int rs[2];pipe(rs);double t0=now();
    if(fork()==0){                                  /* receiver */
        close(fd[1]);char*b=malloc(CH);uint64_t s=0;ssize_t r;size_t got=0;
        while((r=read(fd[0],b,CH))>0){s+=sum64(b,r&~7);got+=r;}
        write(rs[1],&s,8);_exit(got==total?0:1);}
    close(fd[0]);
    for(size_t o=0;o<total;o+=CH)write(fd[1],src,CH);   /* sender */
    close(fd[1]);wait(NULL);read(rs[0],chk,8);return now()-t0;}
struct mbuf{long mtype;char d[4096];};
static double by_msg(size_t total,uint64_t*chk){
    int q=msgget(IPC_PRIVATE,IPC_CREAT|0600);int rs[2];pipe(rs);double
t0=now();
    if(fork()==0){struct mbuf m;uint64_t s=0;
        for(;;){ssize_t r=msgrcv(q,&m,sizeof m.d,0,0);if(r<0)_exit(2);
                if(m.mtype==2)break;s+=sum64(m.d,r&~7);}
        write(rs[1],&s,8);_exit(0);}
    struct mbuf m;m.mtype=1;
    for(size_t
o=0;o<total;o+=4096){memcpy(m.d,src+(o%CH),4096);msgsnd(q,&m,4096,0);}
    m.mtype=2;msgsnd(q,&m,0,0);wait(NULL);read(rs[0],chk,8);
    msgctl(q,IPC_RMID,0);return now()-t0;}
```

Raw output (best of five runs):

```
 size | pipe                | shared memory+semaphore | message queue (4 KB)
  1MB |     0.3 ms   3074 MB/s |     0.3 ms   3340 MB/s |     0.6 ms   1628 MB/s checksums
equal
 10MB |     2.2 ms   4548 MB/s |     1.6 ms   6287 MB/s |     5.4 ms   1868 MB/s checksums
equal
100MB |    20.1 ms   4976 MB/s |    14.4 ms   6967 MB/s |    53.7 ms   1863 MB/s checksums
equal
```

A first version of the benchmark printed CHECKSUM MISMATCH for the message queue. The cause was a bug in the test itself: the message sender kept re-sending the first 4 KB of the source buffer, while the other two methods sent the whole 64 KB buffer, so the checksums legitimately differed. The sender was corrected to cycle through the buffer (`src + (o % CH)`), after which all three checksums matched. The timings in the table are from the corrected program.

B.2 Q5: generalised bounded buffer (`q5_multi.c`)

This version takes the number of producers and consumers as an argument, gives every item a unique identifier, and counts how often each identifier is consumed. With the second argument set to 0 the mutex is skipped.

```
#define SIZE 5
#define ITEMS 1000
int buffer[SIZE], in = 0, out = 0, NP, NC, USE_MUTEX;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0; int seen[8*ITEMS+8];
static void L(void){ if(USE_MUTEX) pthread_mutex_lock(&lock); }
static void U(void){ if(USE_MUTEX) pthread_mutex_unlock(&lock); }
void *producer(void *arg){ long id = (long)arg;
```

```
    for (int item = 1; item <= ITEMS; item++) { int v = id*ITEMS + item;
        sem_wait(&empty); L();
        buffer[in] = v; if(!USE_MUTEX) sched_yield();   /* widen the race
window */
        in = (in + 1) % SIZE; U(); sem_post(&full); }
    return NULL; }
void *consumer(void *arg){ long share = (long)arg;
    for (long k = 0; k < share; k++) {
        sem_wait(&full); L();
        int v = buffer[out]; if(!USE_MUTEX) sched_yield();
        out = (out + 1) % SIZE; U(); sem_post(&empty);
        __atomic_fetch_add(&consumed_sum, v, __ATOMIC_RELAXED);
        __atomic_fetch_add(&seen[v], 1, __ATOMIC_RELAXED); }
    return NULL; }
```

After all threads have joined, the program compares `consumed_sum` with the expected sum and counts identifiers seen zero times (lost) and more than once (duplicated). **Raw output:**

```
P=C=1 mutex=1: consumed 1000, sum=500500 expected=500500, never-consumed=0,
consumed-twice=0 -> CORRECT
P=C=4 mutex=1: consumed 4000, sum=8002000 expected=8002000, never-consumed=0,
consumed-twice=0 -> CORRECT
100 runs, 4P+4C, mutex on : 100 CORRECT
P=C=1 mutex=0: consumed 1000, sum=500500 expected=500500, never-consumed=0,
consumed-twice=0 -> CORRECT
P=C=4 mutex=0: consumed 4000, sum=8013835 expected=8002000, never-consumed=1605,
consumed-twice=1603 -> WRONG
P=C=4 mutex=0: consumed 4000, sum=7989232 expected=8002000, never-consumed=1017,
consumed-twice=1016 -> WRONG
100 runs, 4P+4C, mutex off: 100 WRONG
```

B.3 Q5: seat-booking lock experiment (`q5_lock.c`)

```
#define TRAINS 50
#define COACHES 10
#define SEATS 72
static unsigned char seat[TRAINS][COACHES][SEATS];
static pthread_mutex_t gl = PTHREAD_MUTEX_INITIALIZER, tl[TRAINS],
cl[TRAINS][COACHES];
static int book(int t, int c){              /* check-then-act on one coach
*/
    for (int s = 0; s < SEATS; s++)
        if (!seat[t][c][s]) {
            if (IOUS) usleep(IOUS);          /* experiment B: 500 us
database confirm */
            seat[t][c][s] = 1; return 1; }
    return 0; }
static void *agent(void *a){ long id = (long)a; uint64_t x =
88172645463325252ULL ^ (id*7919+1); long n = 0;
    for (int i = 0; i < OPS; i++) {
        x ^= x << 13; x ^= x >> 7; x ^= x << 17;
        int t = x % TRAINS, c = (x >> 20) % COACHES;
        if      (mode == 1) pthread_mutex_lock(&gl);
        else if (mode == 2) pthread_mutex_lock(&tl[t]);
        else if (mode == 3) pthread_mutex_lock(&cl[t][c]);
        n += book(t, c);
        if      (mode == 1) pthread_mutex_unlock(&gl);
        else if (mode == 2) pthread_mutex_unlock(&tl[t]);
        else if (mode == 3) pthread_mutex_unlock(&cl[t][c]);
```

```
    }
    ok[id] = n; return NULL; }
```

Mode 0 is no lock, 1 one global lock, 2 one lock per train and 3 one lock per coach. After the run the program adds up the tickets the agents believe they sold and counts the seats actually marked occupied; any difference is a seat sold twice. **Raw output of experiment B (500 us inside the lock, 64 agents, 50 bookings each, five runs):**

```
one global lock     1.975 s ; 2.023 s ; 2.001 s ; 2.005 s ; 1.984 s
one lock per train  0.064 s ; 0.070 s ; 0.067 s ; 0.066 s ; 0.071 s
one lock per coach  0.036 s ; 0.037 s ; 0.038 s ; 0.038 s ; 0.040 s
```

B.4 Q5: cost of one counter update (q5_counter.c)

```
static atomic_long ac; static long mc, sc; static atomic_flag sp =
ATOMIC_FLAG_INIT;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static void *fa(void*a){for(long i=0;i<N;i++) atomic_fetch_add(&ac,1);
return 0;}
static void *fm(void*a){for(long
i=0;i<N;i++){pthread_mutex_lock(&m);mc++;pthread_mutex_unlock(&m);} return
0;}
static void *fs(void*a){for(long i=0;i<N;i++){
    while(atomic_flag_test_and_set_explicit(&sp,memory_order_acquire));   /*
spin */
    sc++;
    atomic_flag_clear_explicit(&sp,memory_order_release);} return 0;}
```

Raw output:

```
atomic_fetch_add   1 threads:    6.9 ns/update, CPU time 0.007 s for wall 0.007 s,
final=1000000
atomic_fetch_add   4 threads:    6.8 ns/update, CPU time 0.027 s for wall 0.027 s,
final=4000000
atomic_fetch_add  16 threads:    6.8 ns/update, CPU time 0.108 s for wall 0.109 s,
final=16000000
spin lock          1 threads:    9.1 ns/update, CPU time 0.009 s for wall 0.009 s, final=1000000
spin lock          4 threads:   23.1 ns/update, CPU time 0.092 s for wall 0.093 s, final=4000000
spin lock         16 threads:   64.9 ns/update, CPU time 1.029 s for wall 1.038 s, final=16000000
mutex              1 threads:   23.5 ns/update, CPU time 0.023 s for wall 0.023 s, final=1000000
mutex              4 threads:   25.0 ns/update, CPU time 0.100 s for wall 0.100 s, final=4000000
mutex             16 threads:   23.8 ns/update, CPU time 0.374 s for wall 0.382 s, final=16000000
```

B.5 Q6: timing the safety algorithm (q6_bench.c, core)

```
static int safe(int n,int m,int*avail,int*alloc,int*need,int*work,char*fin){
   memcpy(work,avail,m*sizeof(int)); memset(fin,0,n); int count=0;
   while(count<n){ int found=0;
      for(int p=0;p<n;p++){ if(fin[p]) continue; int ok=1;
         for(int r=0;r<m;r++) if(need[p*m+r]>work[r]){ok=0;break;}
         if(ok){ for(int r=0;r<m;r++) work[r]+=alloc[p*m+r]; fin[p]=1; count++; found=1; } }
      if(!found) return 0; }
   return 1; }
/* worst case: every process holds 1 unit of each resource and needs (n-1-i) units of
```

```
 resource 0, so process n-1 can finish first, then n-2, ... (n passes).
 best case: every need is 0, so all processes finish in the first pass. */
```

Raw output:

```
   n   m | worst case   best case    | checks/s(worst)
  10   3 |      0.1 us      0.0 us | 8384941  (all safe=1)
 100   3 |      8.9 us      0.4 us | 112414  (all safe=1)
1000   3 |    715.0 us      3.3 us | 1399  (all safe=1)
4000   3 |  10791.2 us     13.3 us | 93  (all safe=1)
  10   6 |      0.2 us      0.1 us | 6471623  (all safe=1)
 100   6 |      8.6 us      0.9 us | 116314  (all safe=1)
1000   6 |    689.5 us      7.7 us | 1450  (all safe=1)
4000   6 |  10783.2 us     30.9 us | 93  (all safe=1)
```